

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 1 :

rappel, instructions de bases du langage Python

Par:

Pr. Mohamed-Amine Chadi

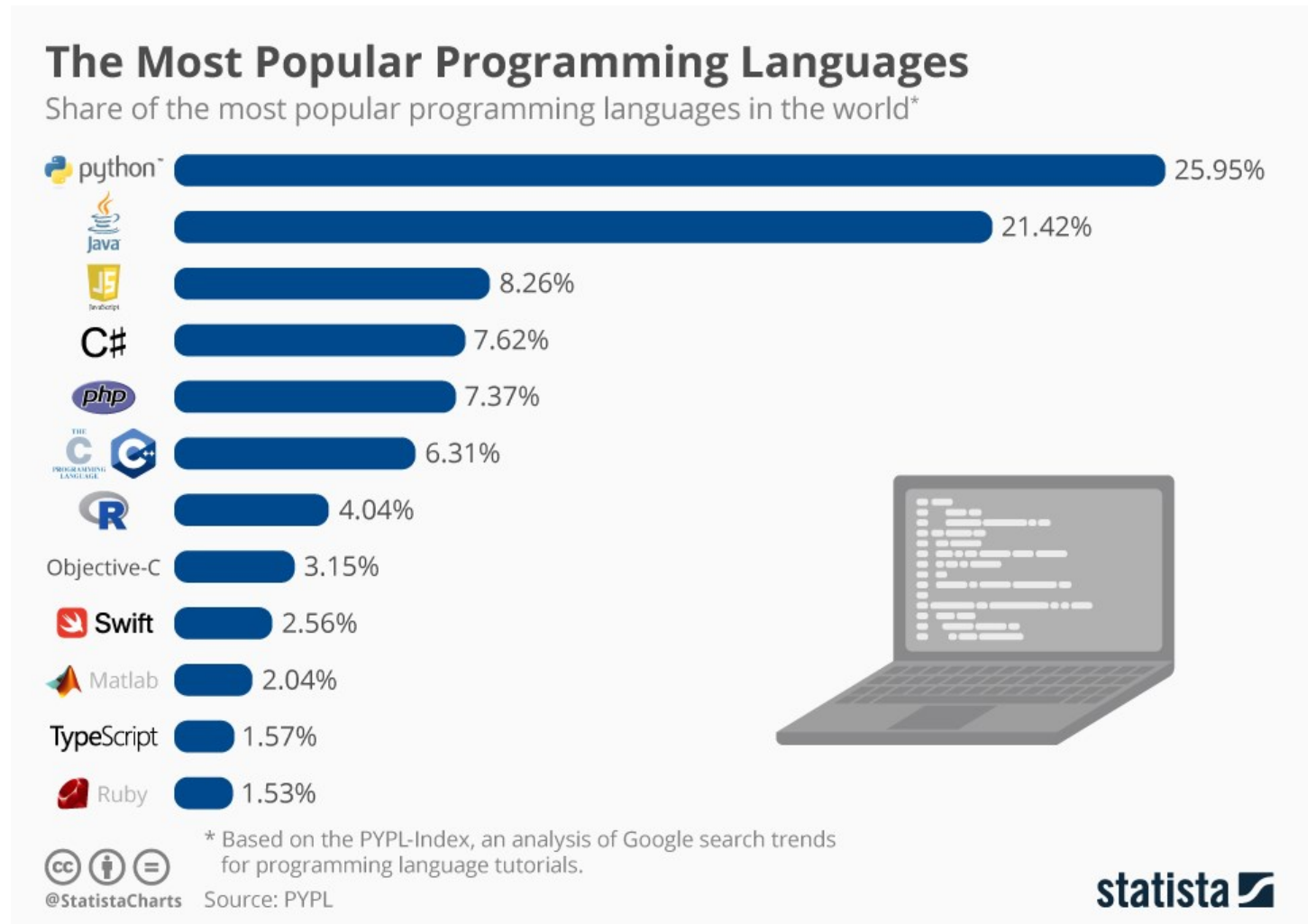
Email: m.chadi@uca.ac.ma

Pourquoi Python

**Populaire, haut-niveau,
à usage général, gratuit et open-source**

Le plus populaire depuis 2019

La popularité aide à apprendre le langage rapidement, puisque il y aura beaucoup de tutoriels, blogs, et de gens qui peuvent répondre aux questions de l'apprenti dans les forums (tels que StackOverFlow)



<https://www.statista.com/chart/16567/popular-programming-languages/>

Python, un langage haut niveau

Exemple en langage binaire x86 d'un programme calculant et 4+5

```
01010101 10001001 11100101 10000011 11101100 00010000 11000111 01000101 11110100
00000100 00000000 00000000 00000000 11000111 01000101 11111000 00000101 00000000
00000000 00000000 10001011 01000101 11111000 10001011 01010101 11110100 10001101
00000100 00000010 10001001 01000101 11111100 10001011 01000101 11111100 11001001
11000011
```

Comme le binaire est difficilement lisible, on utilise très souvent la **représentation hexadécimale** (un chiffre de 0 à f représente 4 bits - voir Codes hexadécimaux page 72), voici la représentation du même programme sous cette forme :

```
55 89 e5 83 ec 10 c7 45 f4 04 00 00 00 c7 45 f8 05 00 00 00 8b 45 f8 8b 55 f4 8d 04 02
89 45 fc 8b 45 fc c9 c3
```

C++

```
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

JAVA

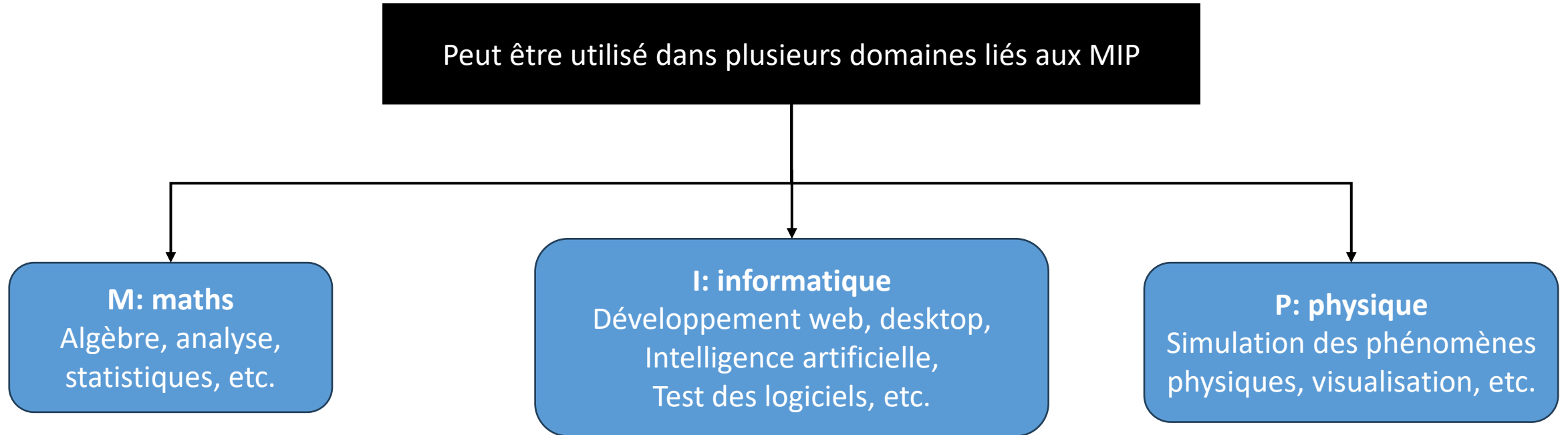
```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

PYTHON

```
>>> print("Hello World!")
Hello World!
```

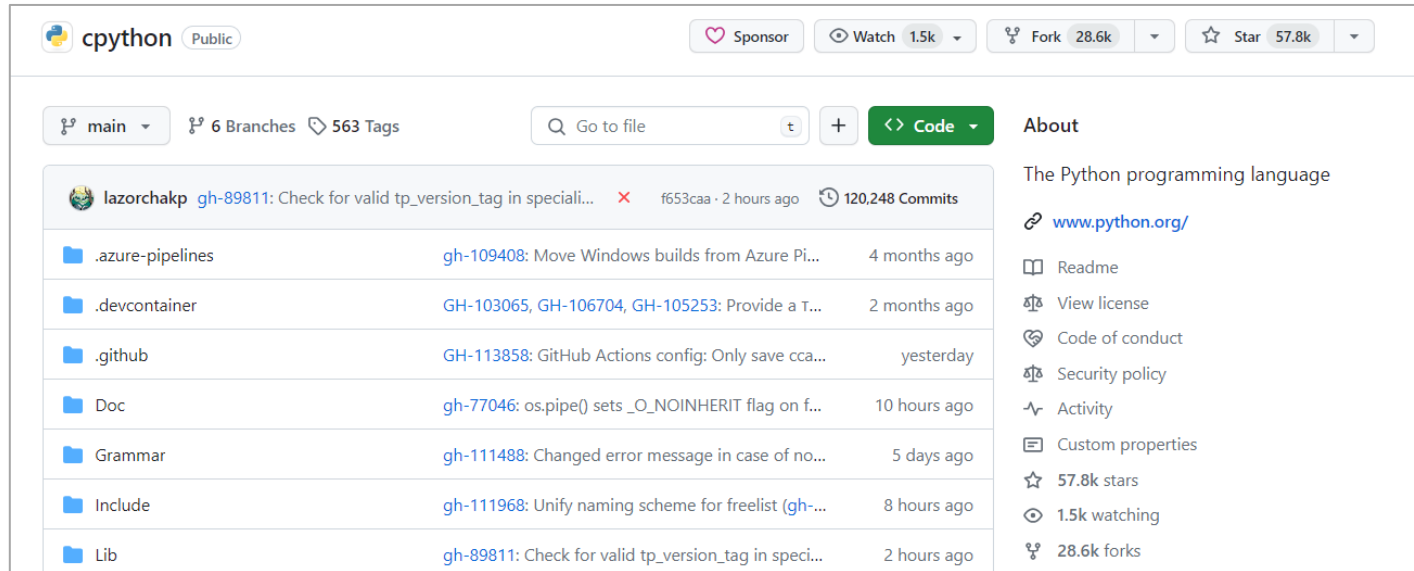
À usage général

Peut-importe la spécialité dans laquelle vous allez continuer → Python va vous aider



Open source

Tout le monde peut voir / consulter le code de Python lui-même

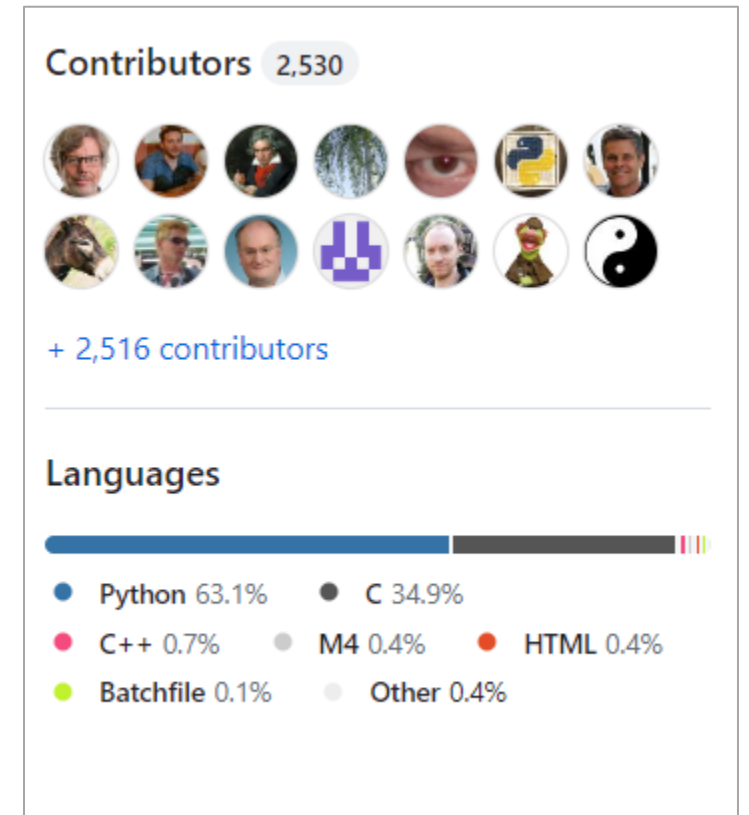


The screenshot shows the GitHub interface for the 'cpython' repository. At the top, it indicates the repository is 'Public' and shows metrics: 1.5k Watchers, 28.6k Forks, and 57.8k Stars. Below this, there's a search bar and a 'Go to file' button. The main content area displays a list of recent commits, including one by 'lazorchakp' and another by 'f653caa'. To the right, the 'About' section describes it as 'The Python programming language' and provides links to the website, README, license, and other resources.

<https://github.com/python/cpython>



Guido Van Rossum



This section displays the 'Contributors' and 'Languages' for the Python repository. The 'Contributors' section shows a total of 2,530 contributors, with a grid of 14 circular profile pictures and a link to '+ 2,516 contributors'. The 'Languages' section features a horizontal bar chart showing the distribution of code across different languages: Python (63.1%), C (34.9%), C++ (0.7%), M4 (0.4%), HTML (0.4%), Batchfile (0.1%), and Other (0.4%).

Language	Percentage
Python	63.1%
C	34.9%
C++	0.7%
M4	0.4%
HTML	0.4%
Batchfile	0.1%
Other	0.4%

Exécution

Interprété (Python)

Compilé (C, C++)

Un peu des deux (Java)

Compilé VS interprété

Processus de compilation :

- Le code source est traduit en langage machine ou en un code intermédiaire par un compilateur avant l'exécution.
- Ce processus aboutit à un fichier exécutable autonome ou un binaire qui peut être directement exécuté par le matériel de l'ordinateur.

Exécution :

- Le code compilé s'exécute directement sur le matériel de l'ordinateur, ce qui conduit généralement à une exécution plus rapide.
- L'étape de compilation se produit avant l'exécution du programme.

Exemples de langages compilés :

- C, C++, Fortran.

Avantages :

- Généralement une exécution plus rapide car le code est déjà traduit en langage machine.
- Le binaire compilé peut être distribué sans révéler le code source.

Inconvénients :

- Cycle de développement plus long car la compilation est une étape distincte. (Pour voir le résultat du code, on doit compiler)
- Des fichiers binaires spécifiques peuvent être nécessaires pour chaque système d'exploitation différent.

Processus d'interprétation :

- Le code source est exécuté ligne par ligne ou déclaration par déclaration par un interpréteur.
- L'interpréteur traduit et exécute le code à la volée, sans produire de fichier exécutable autonome.

Exécution

- Le code est exécuté au moment de l'exécution, et les modifications apportées au code peuvent être immédiatement testées sans étape de compilation séparée.

Exemples :

- Python, JavaScript, PHP.

Avantages :

- Cycle de développement plus court car il n'y a pas besoin d'une étape de compilation distincte.
- Le code peut être facilement modifié et testé pendant le développement.

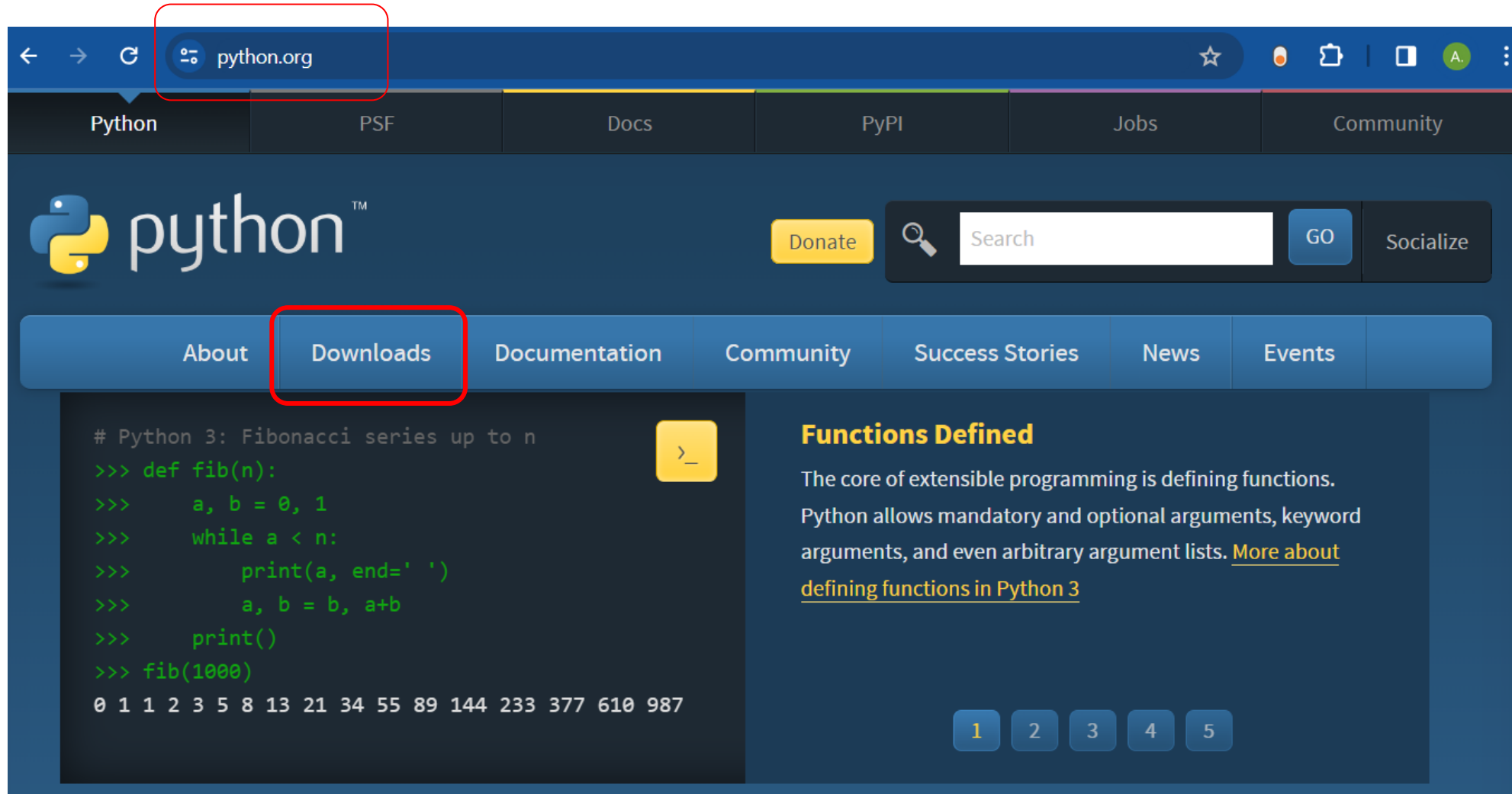
Inconvénients :

- Généralement une exécution plus lente par rapport aux langages compilés.
- Nécessite la présence de l'interpréteur sur le système cible.
- Peut exposer le code source, car il est souvent distribué dans sa forme originale.

Installation

Télécharger Python.exe, version,
Étapes d'installation,
IDLE,
IDE modernes

Télécharger Python.exe



The screenshot shows the Python.org website. The address bar at the top contains 'python.org', which is highlighted with a red box. Below the address bar is a navigation bar with links: Python, PSF, Docs, PyPI, Jobs, and Community. The main header features the Python logo, a 'Donate' button, a search bar with a 'GO' button, and a 'Socialize' button. Below the header is a secondary navigation bar with links: About, Downloads, Documentation, Community, Success Stories, News, and Events. The 'Downloads' link is highlighted with a red box. The main content area is divided into two columns. The left column contains a code snippet for a Fibonacci series generator, with a yellow button containing '>_' next to it. The right column has the heading 'Functions Defined' and a paragraph of text about defining functions in Python, with a link to 'More about defining functions in Python 3'. At the bottom right of the main content area, there are five numbered buttons: 1, 2, 3, 4, and 5.

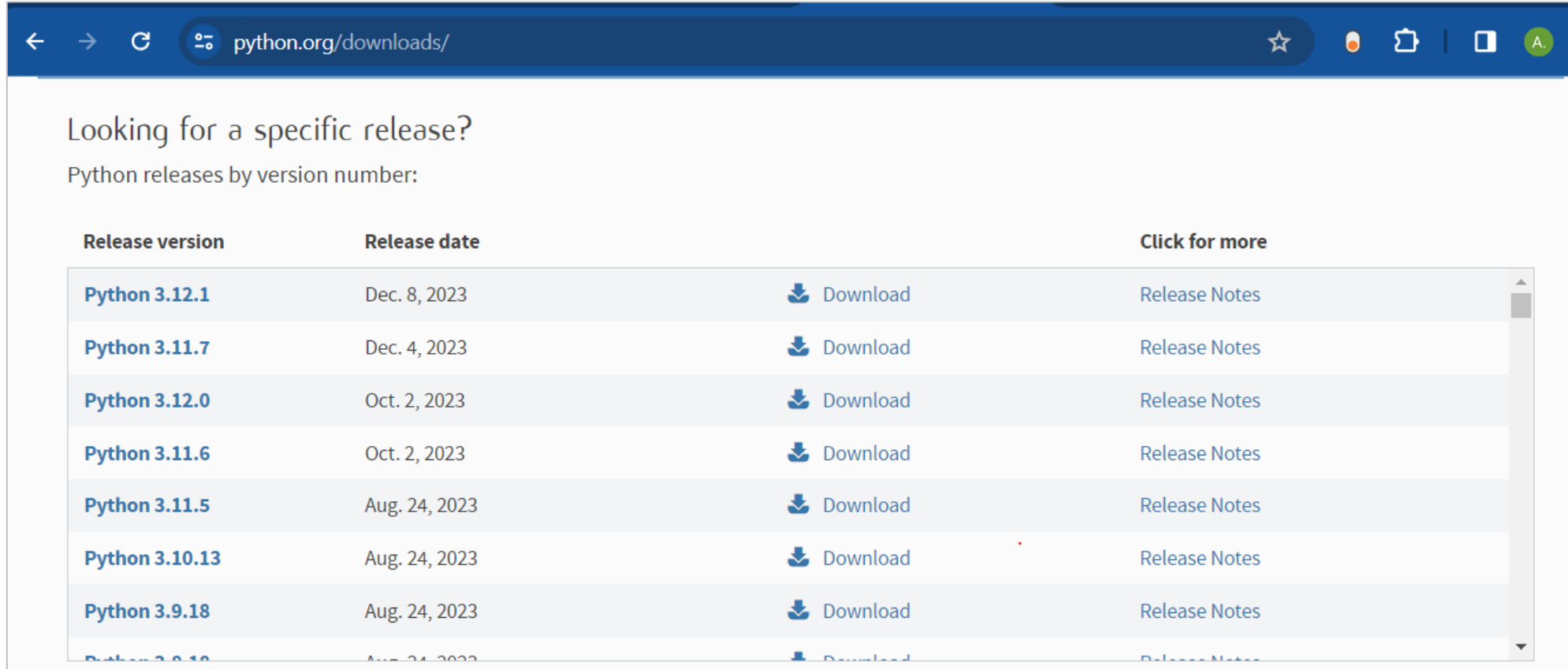
```
# Python 3: Fibonacci series up to n
>>> def fib(n):
>>>     a, b = 0, 1
>>>     while a < n:
>>>         print(a, end=' ')
>>>         a, b = b, a+b
>>>     print()
>>> fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Functions Defined

The core of extensible programming is defining functions. Python allows mandatory and optional arguments, keyword arguments, and even arbitrary argument lists. [More about defining functions in Python 3](#)

1 2 3 4 5

Télécharger et cliquer sur le « .exe »

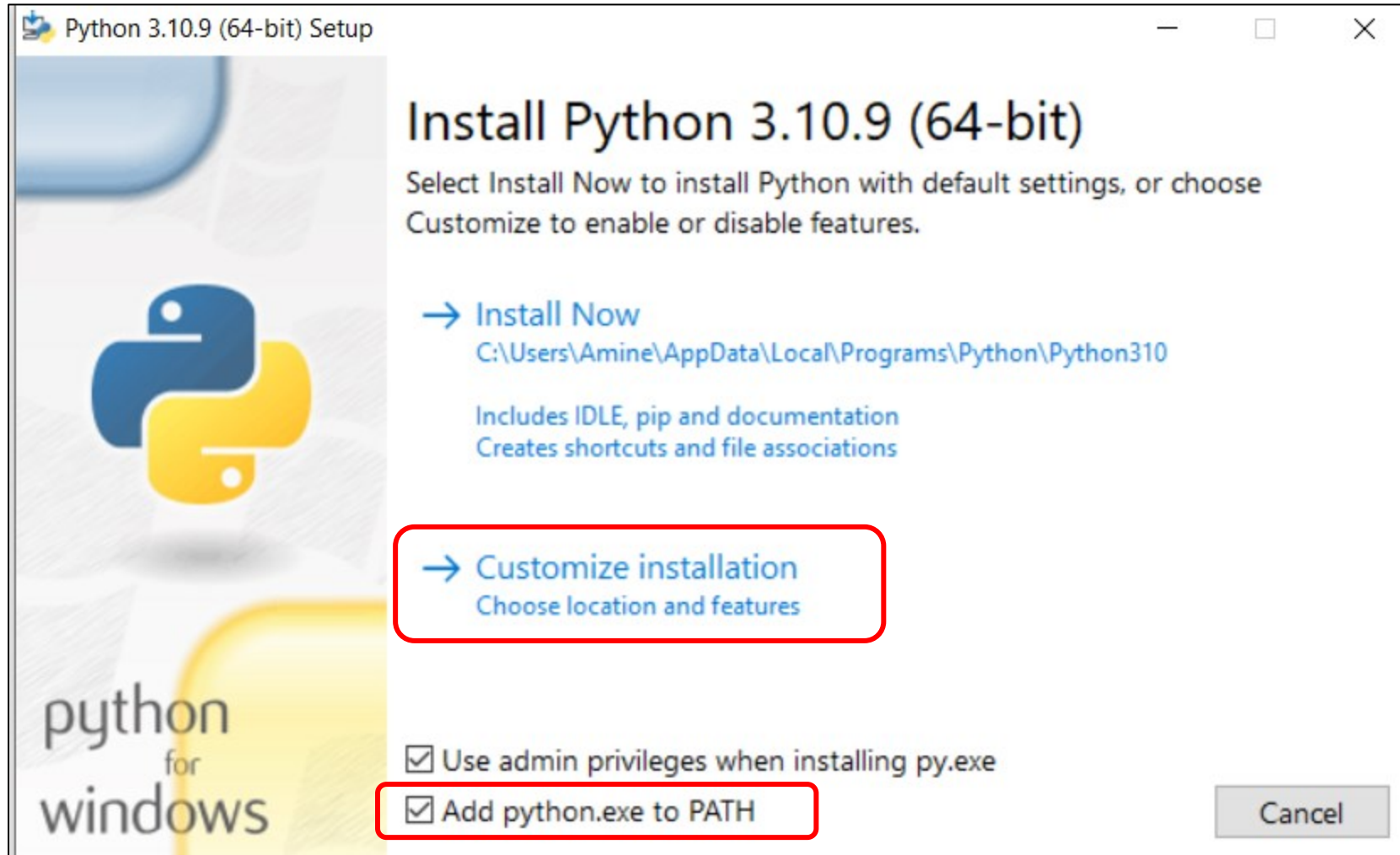


A screenshot of the Python.org website's download page. The browser's address bar shows 'python.org/downloads/'. The page content includes a heading 'Looking for a specific release?' and a subheading 'Python releases by version number:'. Below this is a table with three columns: 'Release version', 'Release date', and 'Click for more'. The table lists several Python versions, including 3.12.1, 3.11.7, 3.12.0, 3.11.6, 3.11.5, 3.10.13, and 3.9.18. Each row has a 'Download' link with a download icon and a 'Release Notes' link.

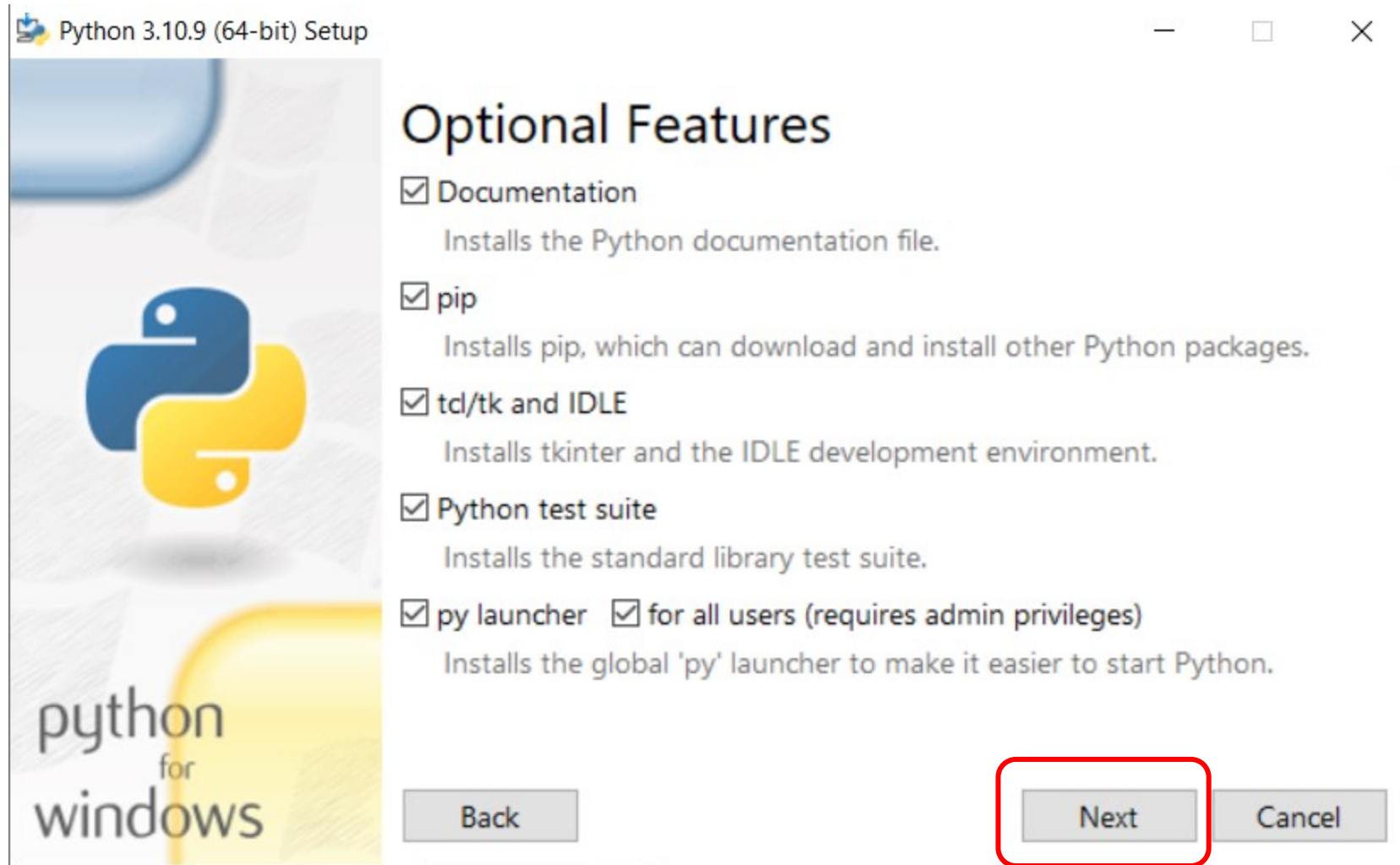
Release version	Release date	Click for more
Python 3.12.1	Dec. 8, 2023	Download Release Notes
Python 3.11.7	Dec. 4, 2023	Download Release Notes
Python 3.12.0	Oct. 2, 2023	Download Release Notes
Python 3.11.6	Oct. 2, 2023	Download Release Notes
Python 3.11.5	Aug. 24, 2023	Download Release Notes
Python 3.10.13	Aug. 24, 2023	Download Release Notes
Python 3.9.18	Aug. 24, 2023	Download Release Notes

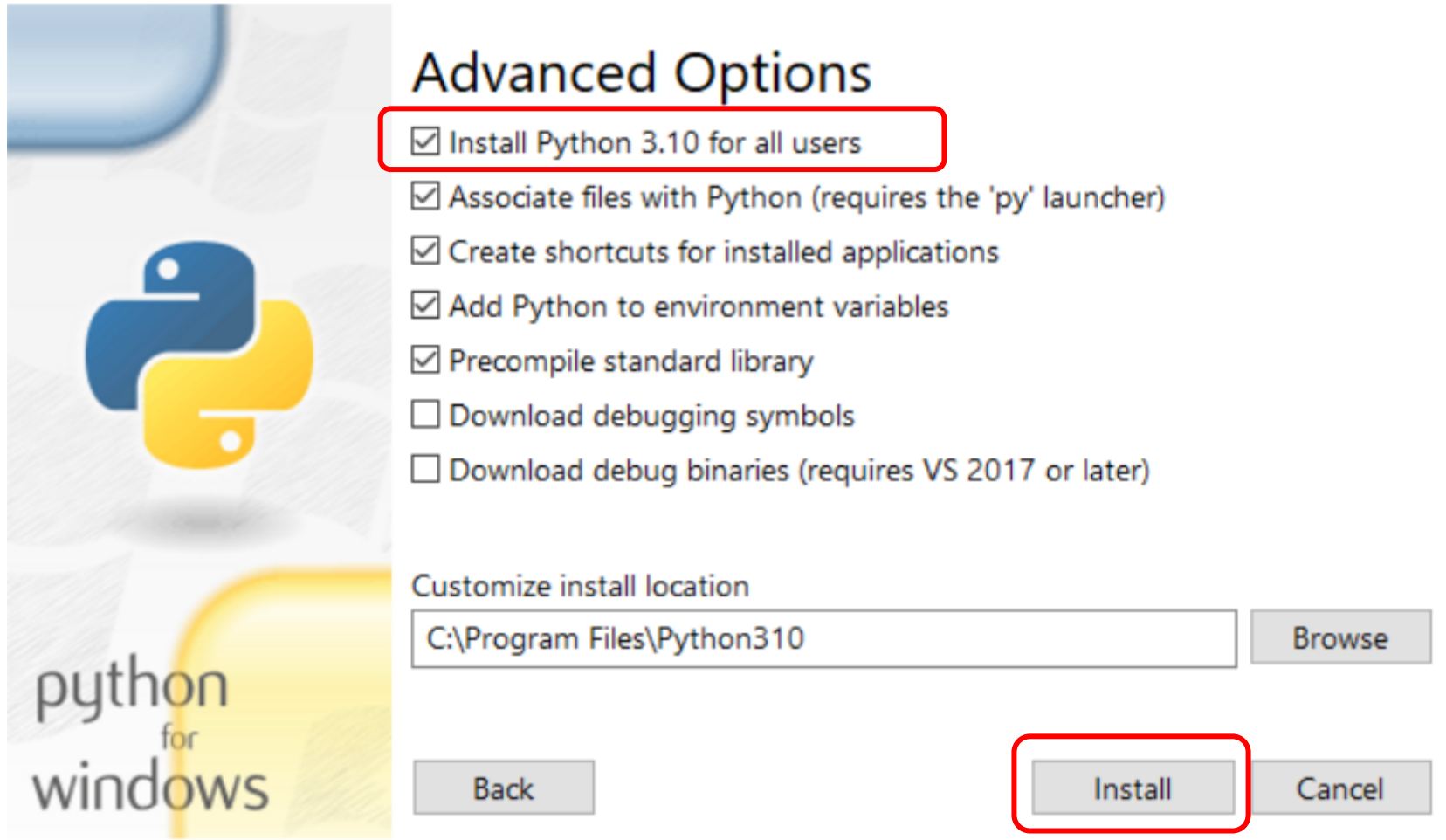
 **python-3.10.9-amd64.exe**

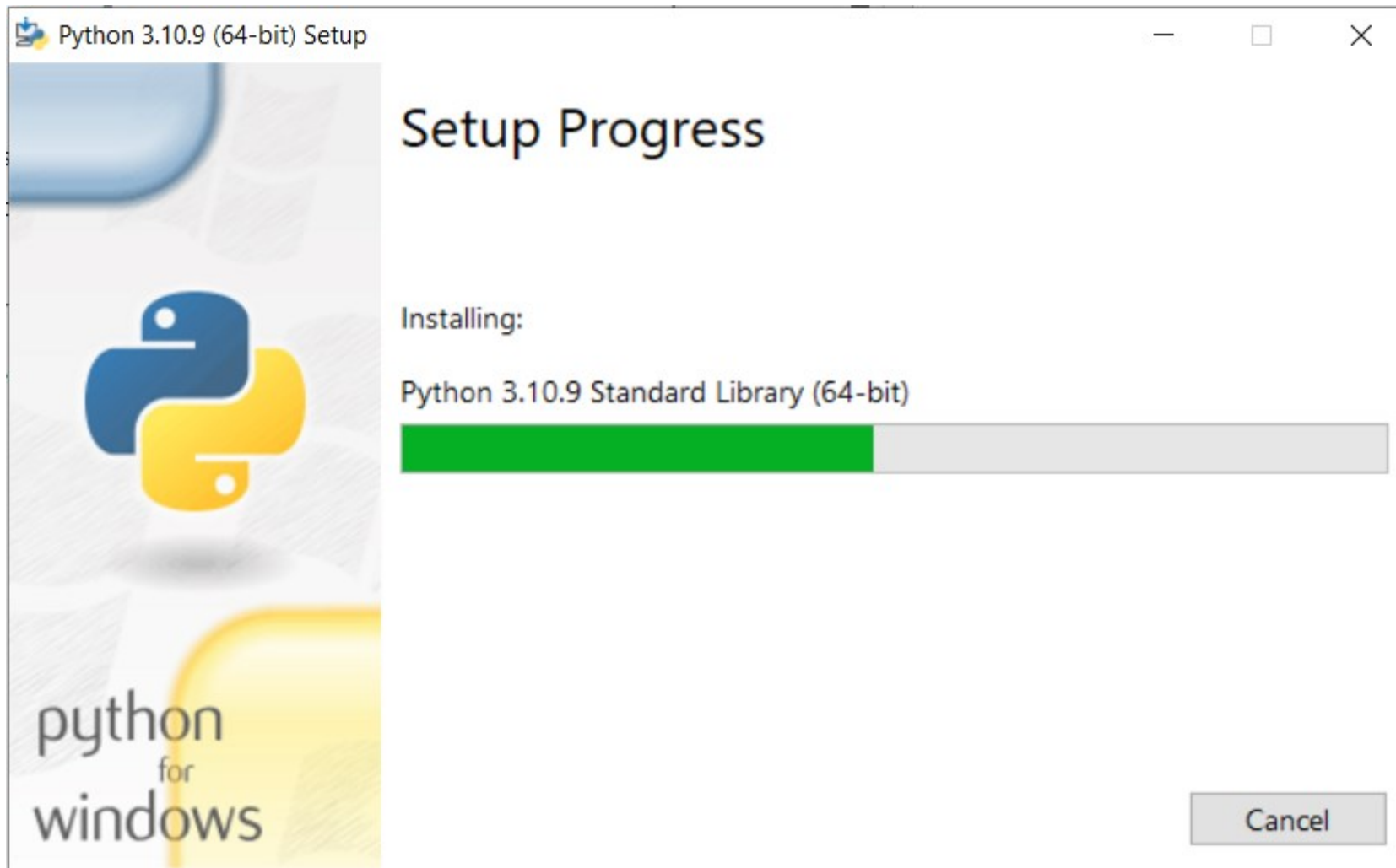
Cocher « Add Python to PATH » Et cliquer sur « Customize installation »

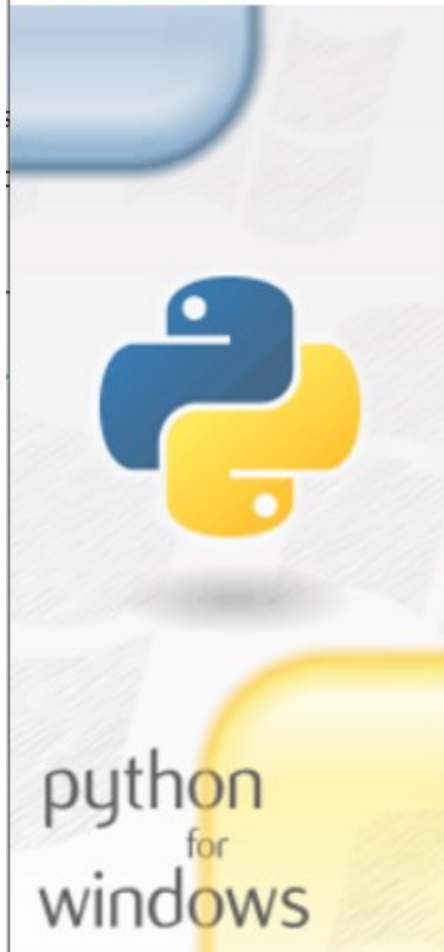


Télécharger et installer Python.exe









Setup was successful

New to Python? Start with the [online tutorial](#) and [documentation](#). At your terminal, type "py" to launch Python, or search for Python in your Start menu.

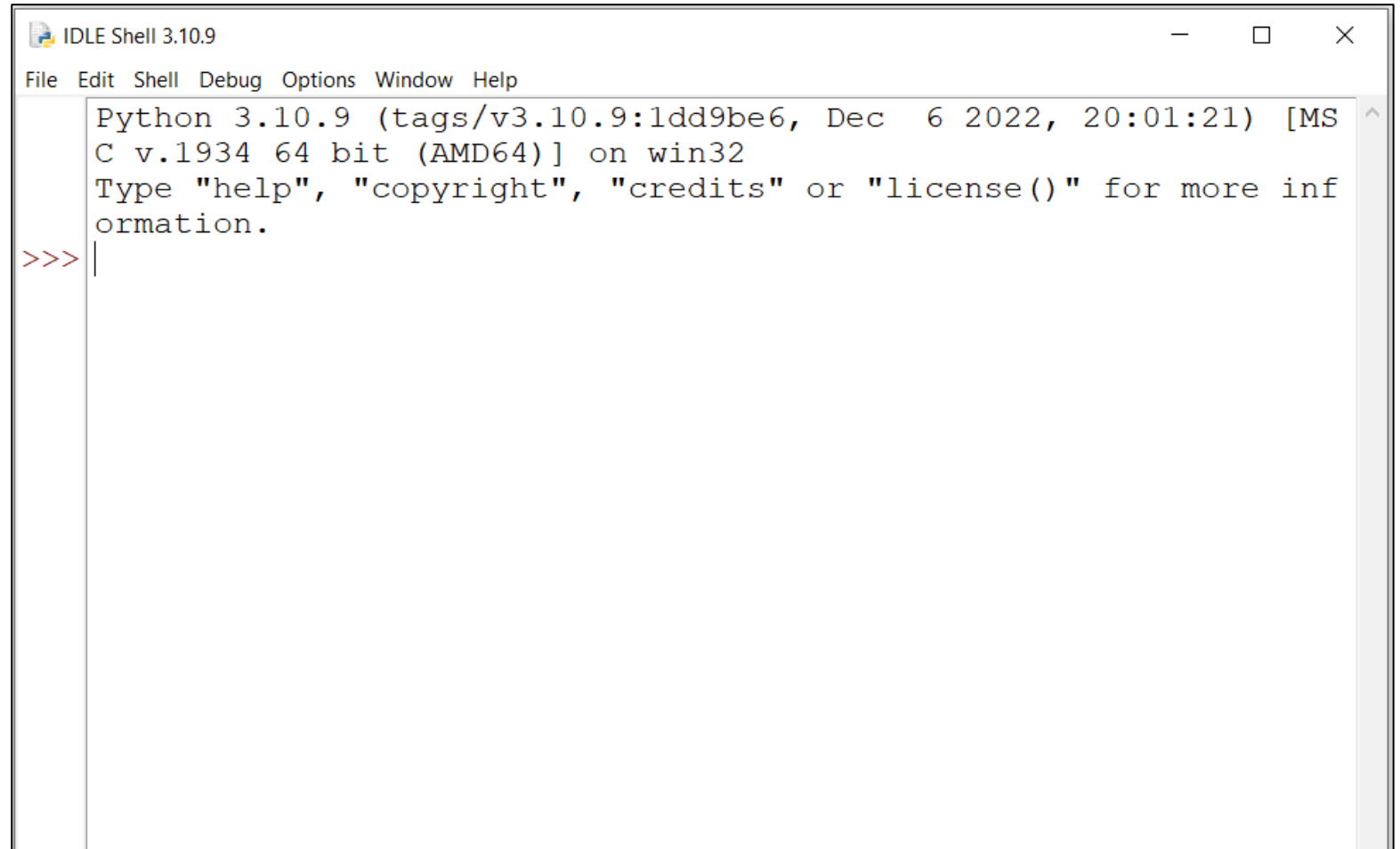
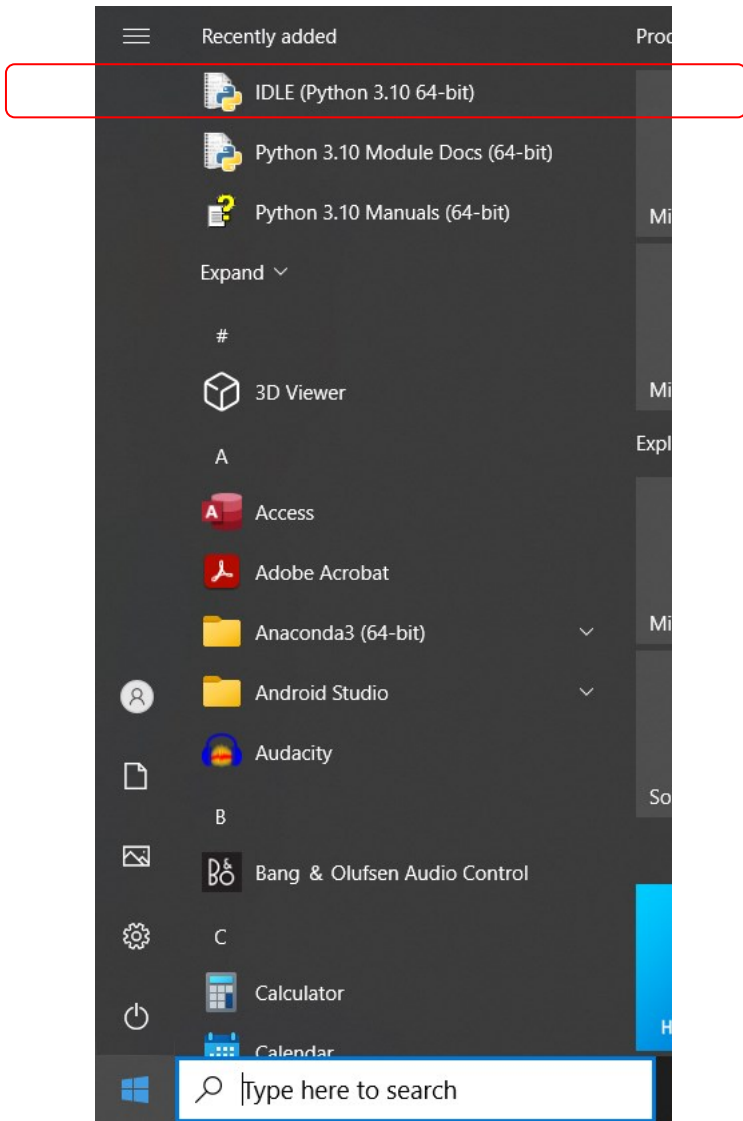
See [what's new](#) in this release, or find more info about [using Python on Windows](#).

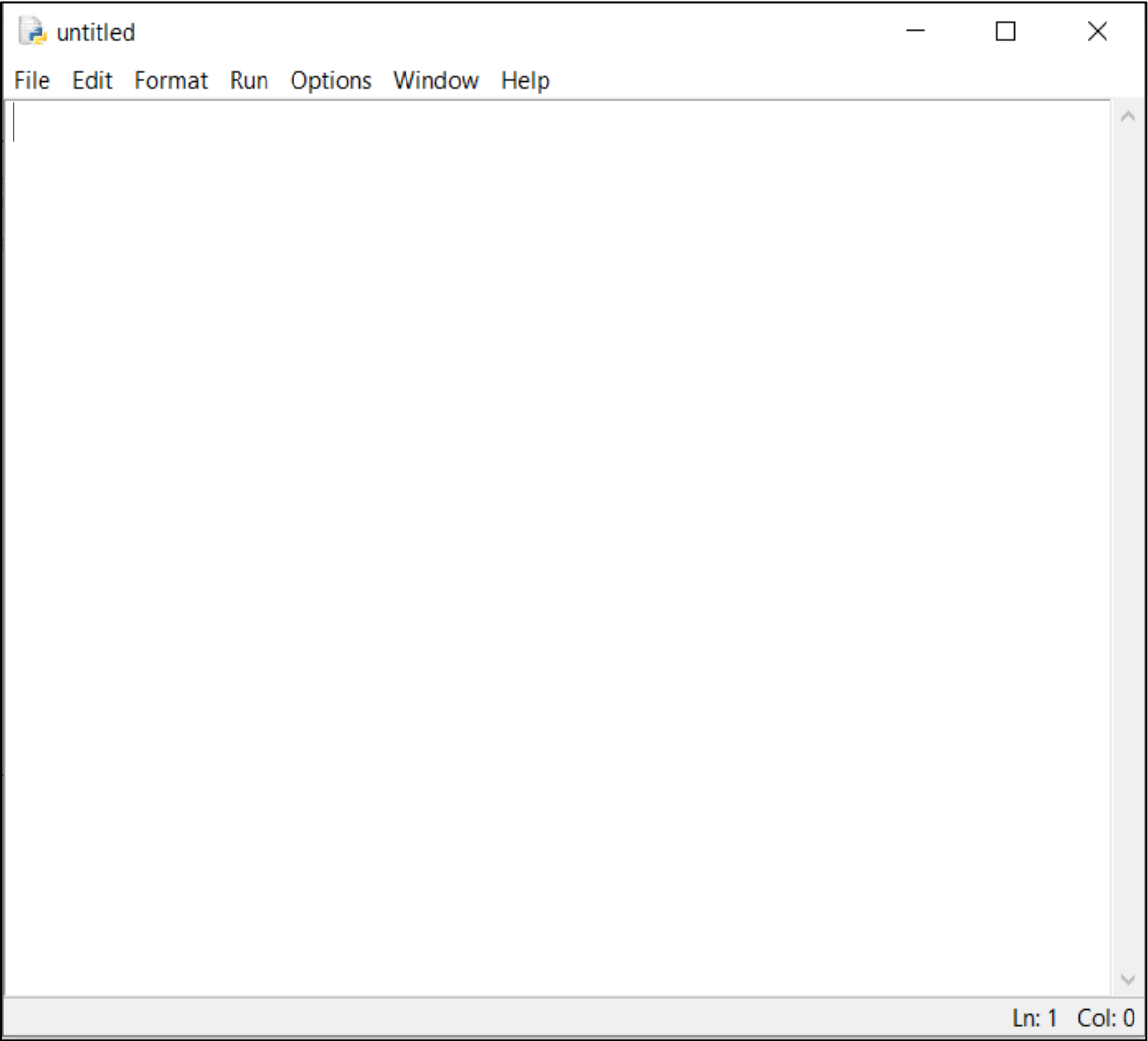
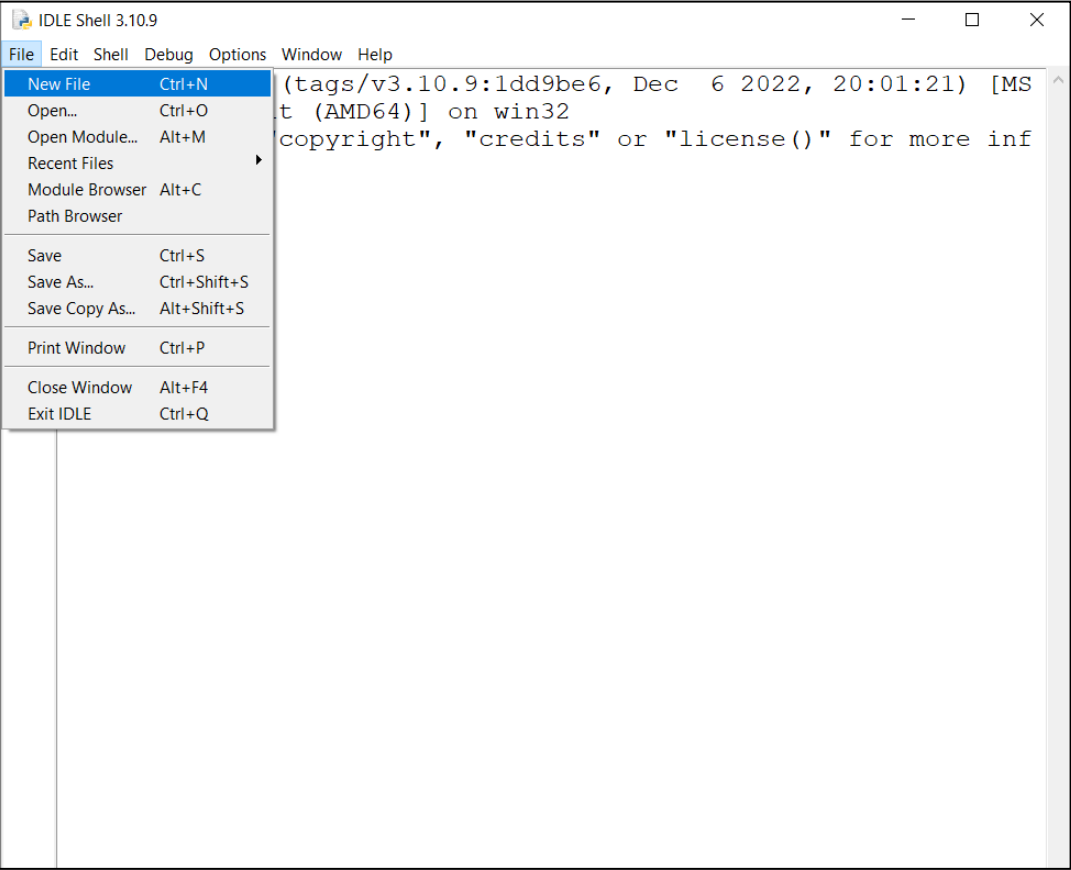


Disable path length limit

Changes your machine configuration to allow programs, including Python, to bypass the 260 character "MAX_PATH" limitation.

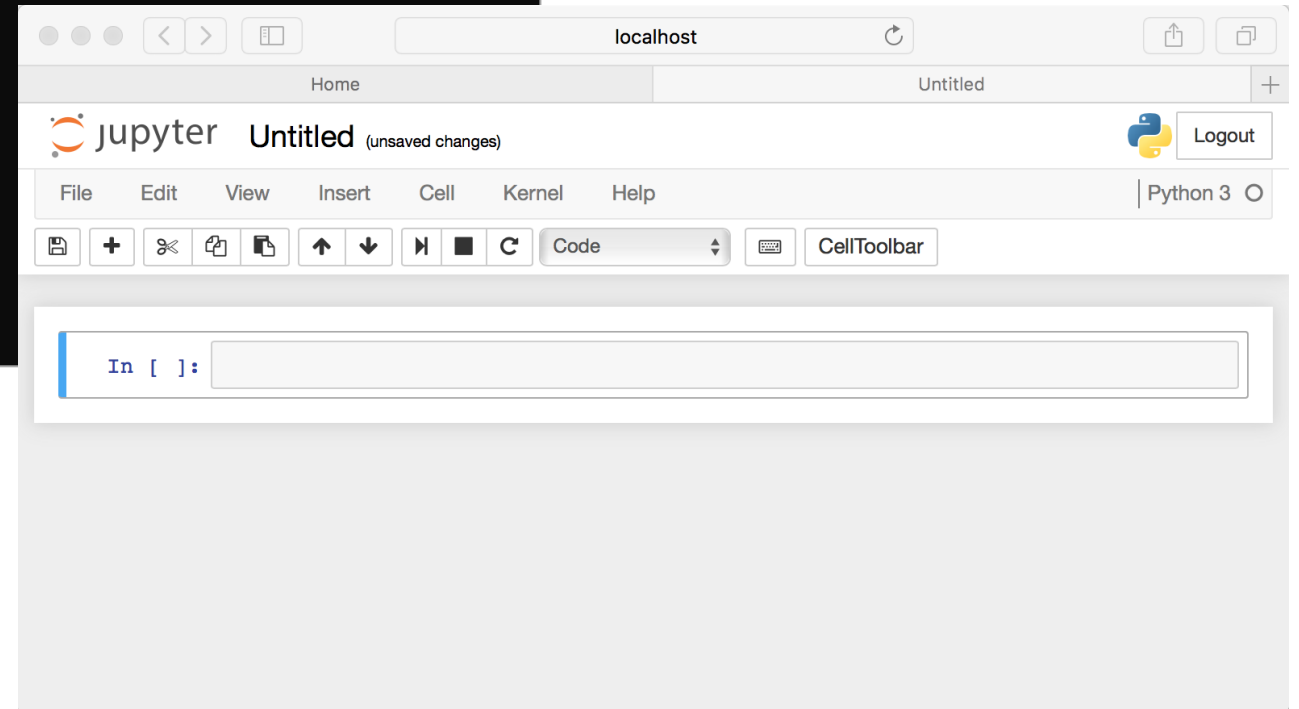
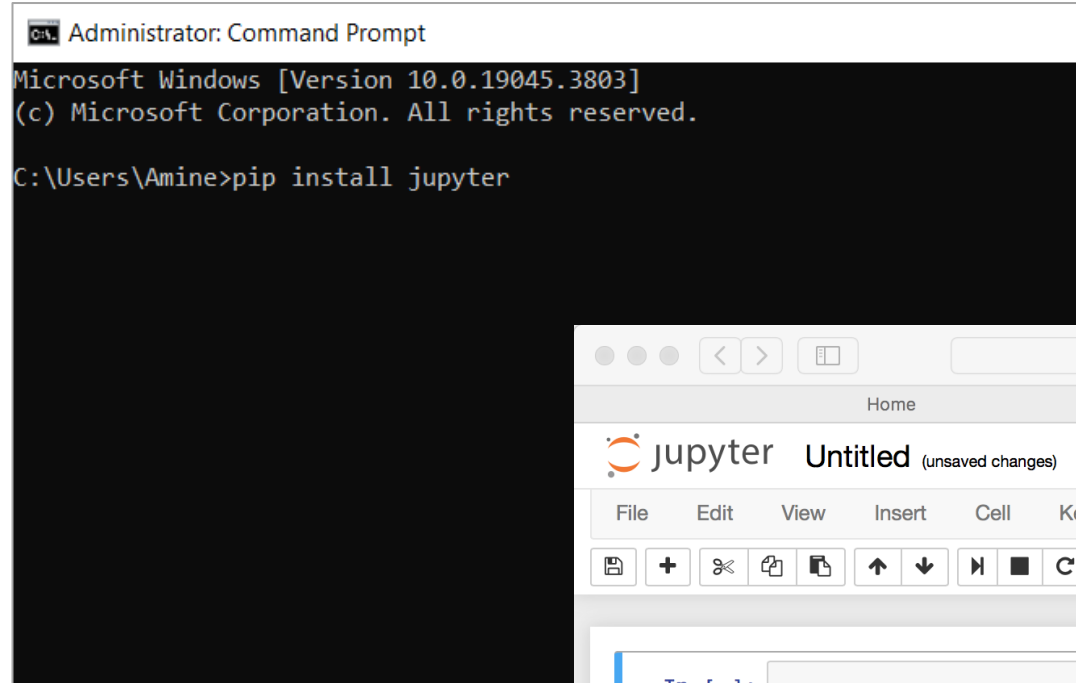
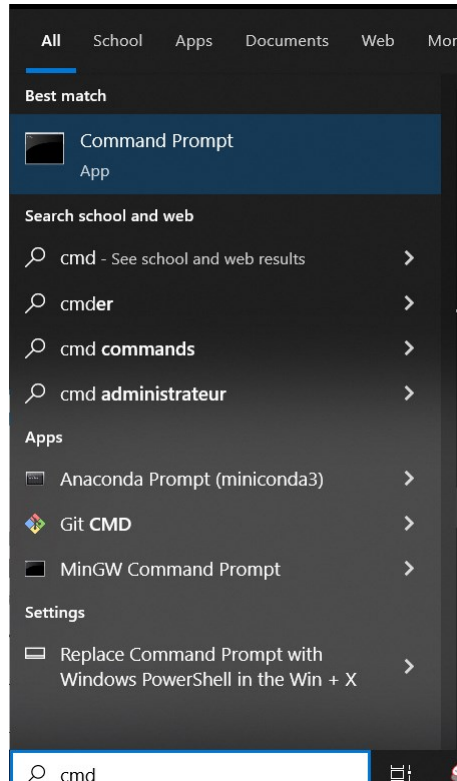
Close





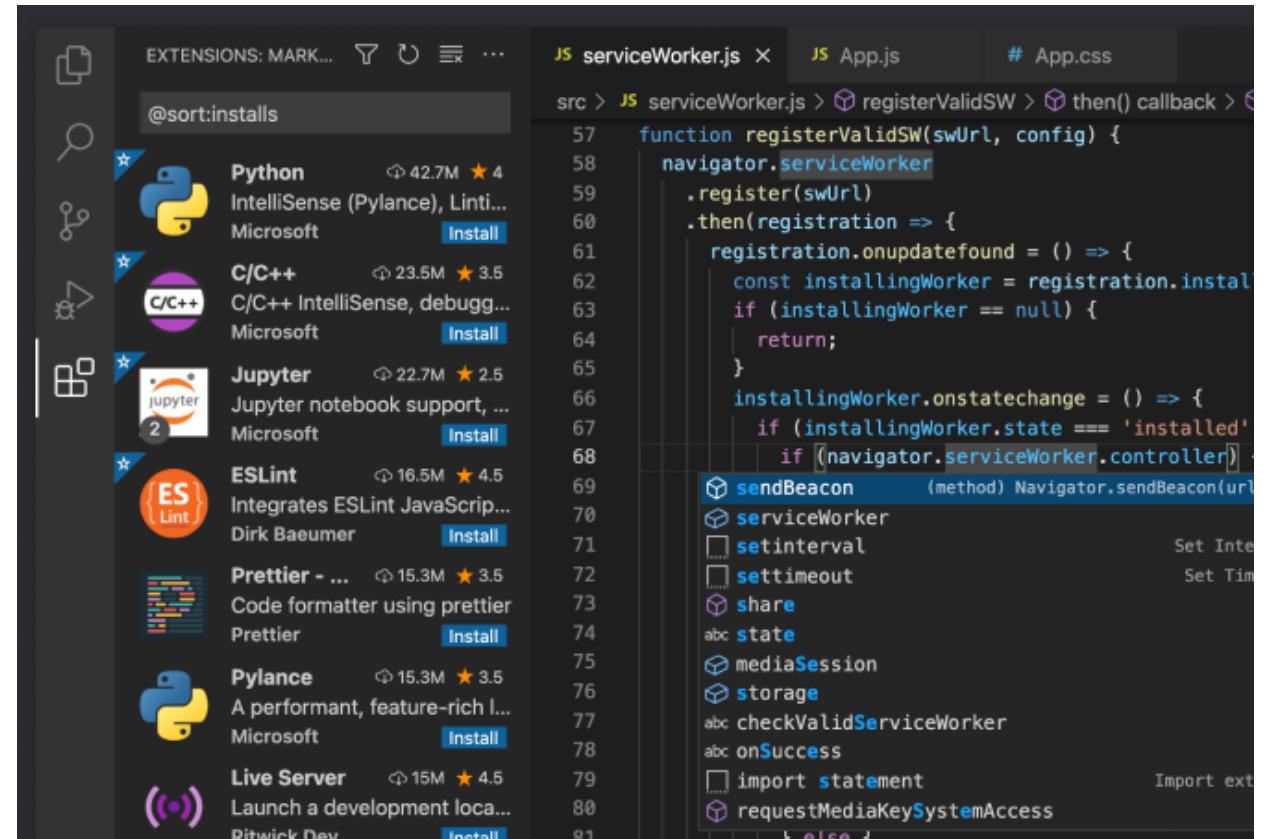
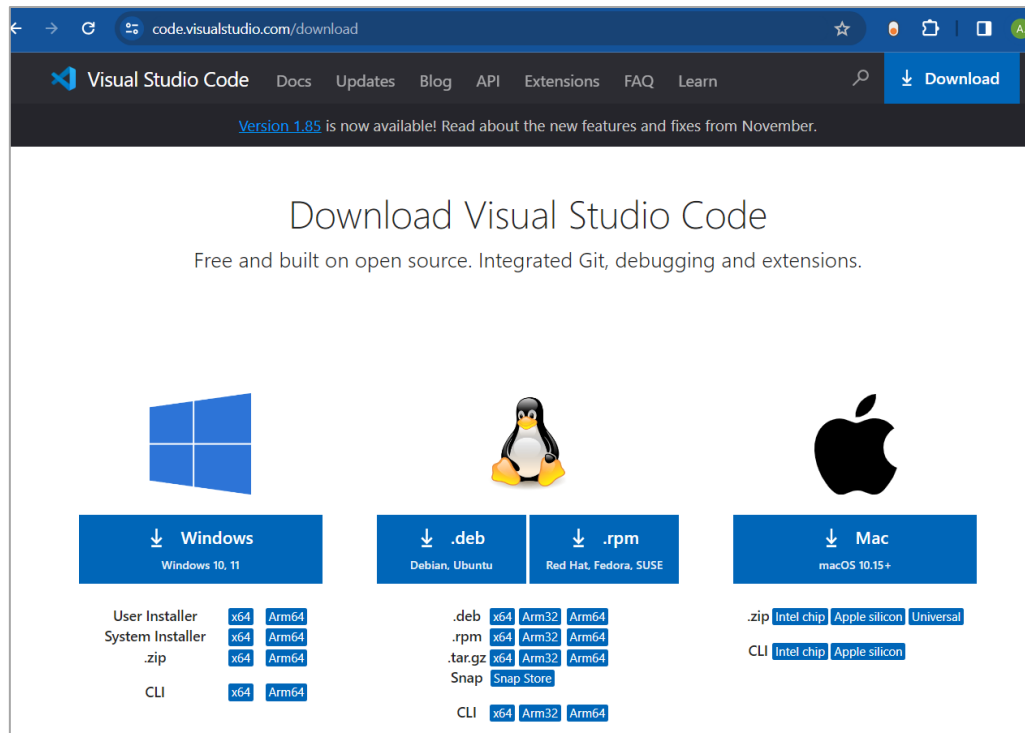
Autres IDE modernes

Jupyter notebook



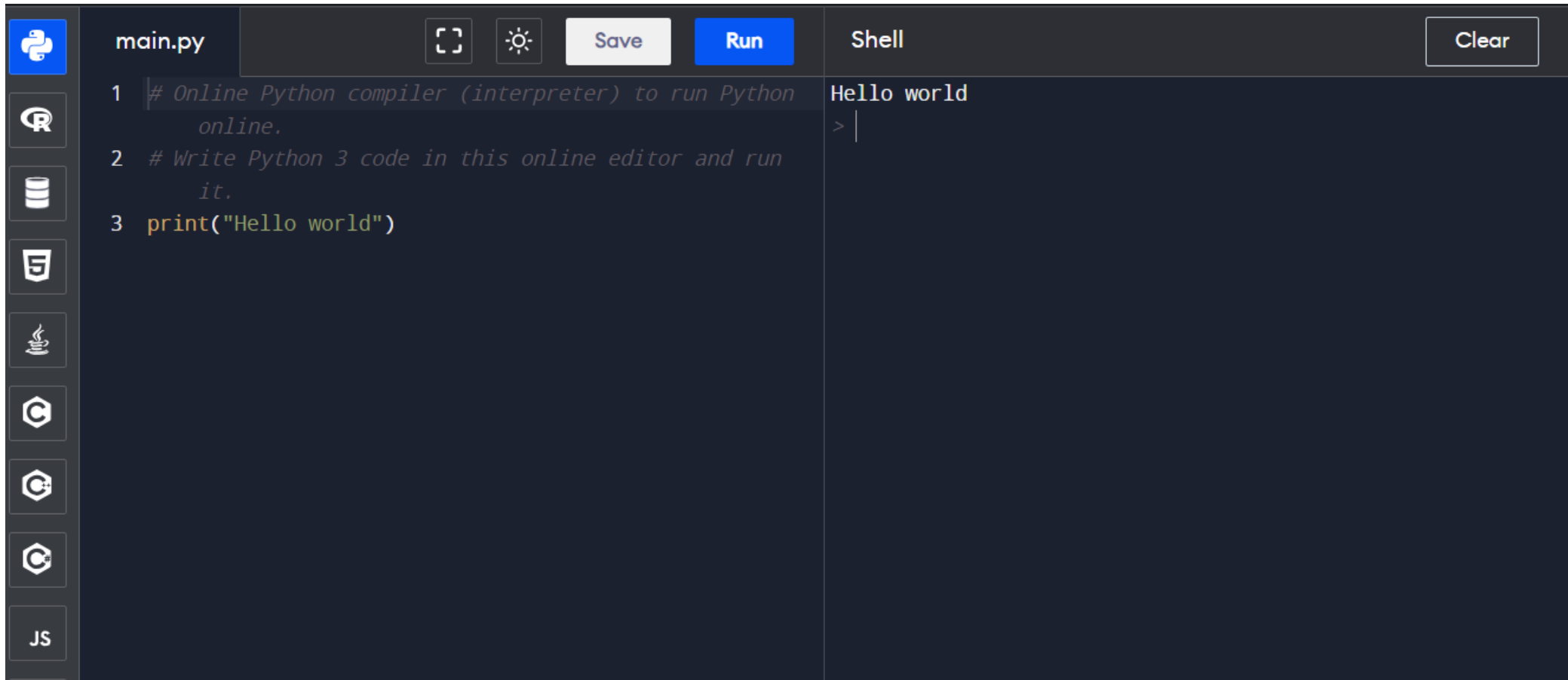
Autres IDE modernes

Visual studio Code



Sinon, IDE en ligne

Pas besoin d'installation



The screenshot shows the Programiz online Python IDE interface. On the left is a vertical sidebar with icons for Python, R, Java, C++, C, JavaScript, and a 'JS' button at the bottom. The main editor area is titled 'main.py' and contains three lines of Python code: a comment about the online compiler, another comment about writing code in the editor, and a `print("Hello world")` statement. Above the code are icons for a code viewer, a sun icon, and buttons for 'Save' and 'Run'. To the right of the code editor is a 'Shell' window showing the output 'Hello world' with a prompt character '>' and a cursor. A 'Clear' button is located in the top right corner of the Shell window.

```
main.py
```

```
1 # Online Python compiler (interpreter) to run Python online.  
2 # Write Python 3 code in this online editor and run it.  
3 print("Hello world")
```

```
Shell
```

```
Hello world  
> |
```

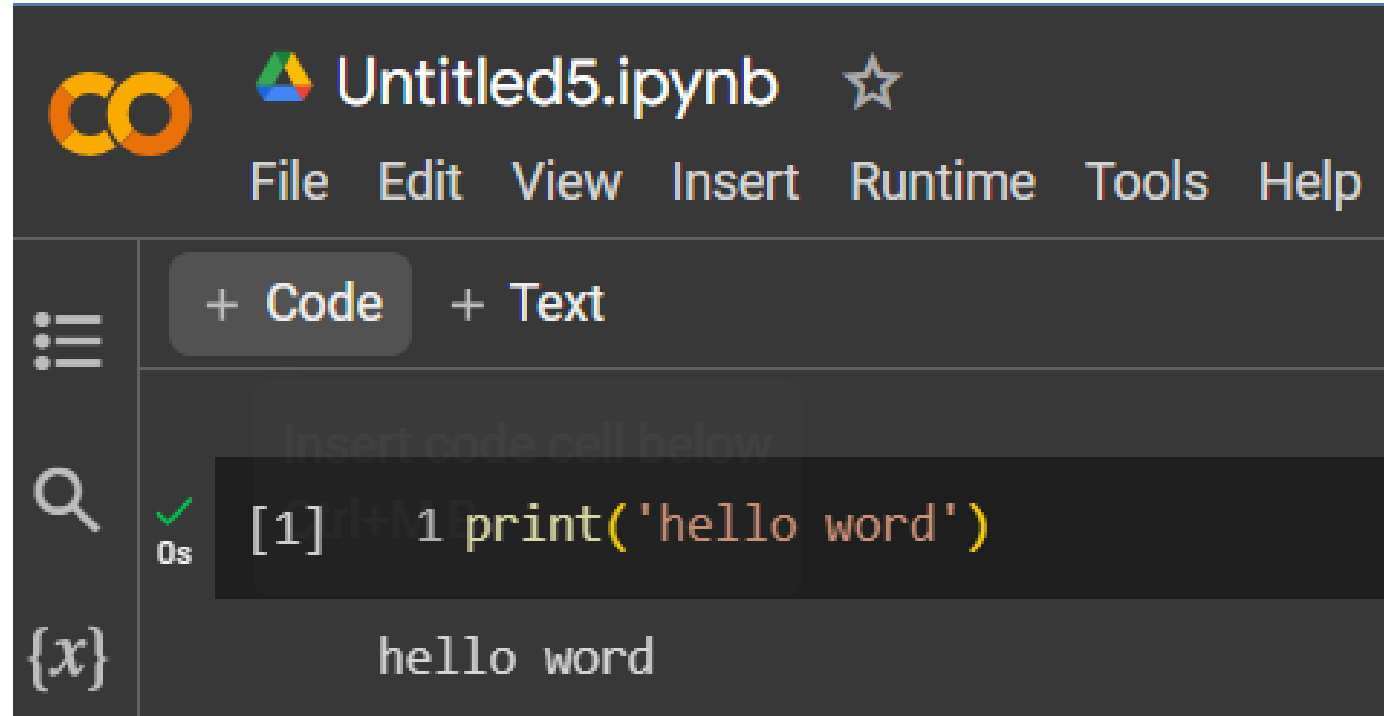
```
Clear
```

Programiz

<https://www.programiz.com/python-programming/online-compiler/>

Sinon, utiliser un IDE en ligne

Pas besoin d'installation



Google colab

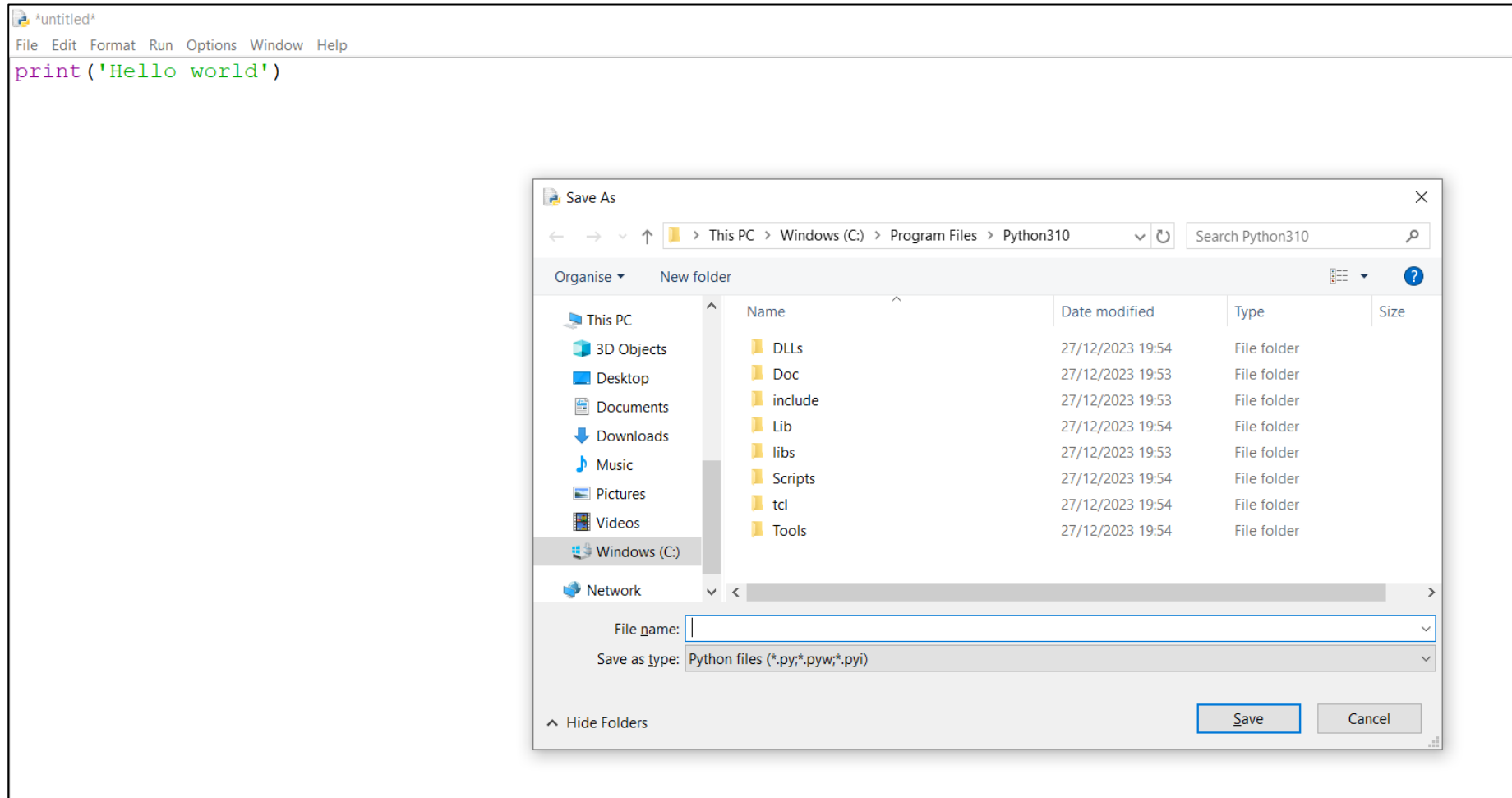
<https://colab.research.google.com/>

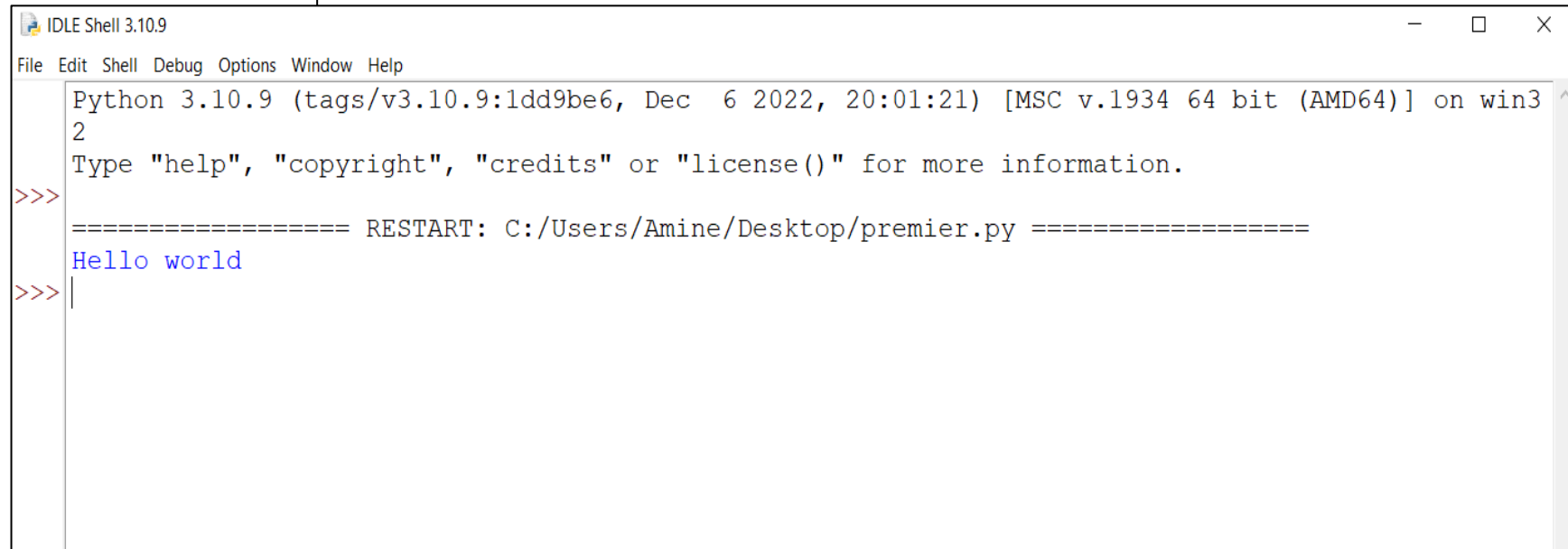
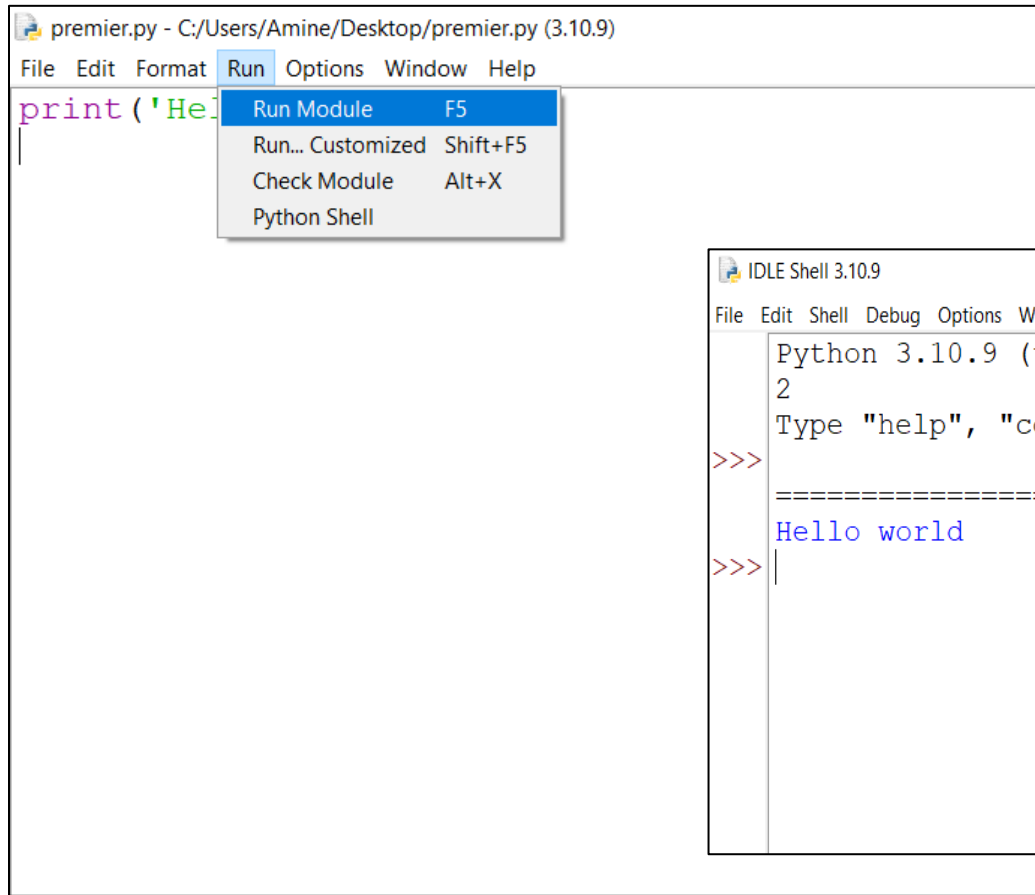
Votre premier programme Python

```
print('hello world')
```

print('Hello word')

Sauvegarder le fichier et exécuter pour voir le résultat





Types de variables

Typage dynamique

et quelques fonctions prédéfinies dans Python (print, type, input, len, etc.)

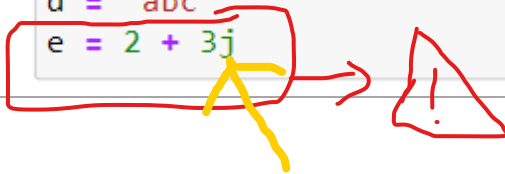
int, float, bool, complex, string

Types des variables

Et opérations

```
▶ # types de variables
...
int    float    bool    string    complex
...

a = 1
b = 2.5
c = True
d = 'abc'
e = 2 + 3j
```



```
▶ print(type(a))
<class 'int'>

▶ print(type(b))
<class 'float'>

▶ print(type(c))
<class 'bool'>

▶ print(type(d))
<class 'str'>

▶ print(type(e))
<class 'complex'>
```

```
a * 3    #ou b * 3
3

c * 3    #True est comme 1
3

d * 3
'abcabcabc'

e * 3
(6+9j)
```

Python est un langage dynamiquement typé, c.à.d., le type des variables est déterminé de manière automatique lors de l'interprétation et l'exécution du programme. Au contraire des langages statiquement typés tel que le langage C, où la déclaration des variables consiste à définir le type de variable et sa valeur

Structures de données

lists, set, dict, tuple, et string

Accès et mutabilité

Définitions

```
a = [1, 2.5, 3, 'mohamed']
print(type(a))

b = {1, 2, 3, 'mohamed', 2, 3}
print(type(b))

c = {1:'a', 2:'b', 3:'c', 4:'mohamed'}
print(type(c))

d = (1, 2, 3, 'mohamed')
print(type(d))
```

```
<class 'list'>
<class 'set'>
<class 'dict'>
<class 'tuple'>
```

```
print(b)

{1, 2, 3, 'mohamed'}
```

- **Tableaux** : structure de données qui peut contenir plusieurs valeurs de même type. Exemple, chaîne de caractère.
- **Listes** : comme le tableau, sauf qu'elle peut contenir plusieurs valeurs de types différents
- **Set** : comme les listes, sauf que l'ordre n'est pas pris en considération (donc pas d'indexation par position), et surtout, il contient des valeurs uniques
- **Tuple** : comme les listes, sauf qu'ils sont immutables (plus de détails dans les slides suivants)
- **Dictionnaire** : clés – valeurs, clés doivent être uniques, l'ordre ici est aussi pas important (donc pas d'indexation par position)

List, set, dictionnaire = mutable



```
'''  
lists, set, dictionnaire      = Mutable  
chaines de caractère et tuple = Immutable  
'''
```

```
a = [1, 2.5, 3, 'mohamed']  
print(type(a))
```

```
b = {1, 2, 3, 'mohamed', 2, 3}  
print(type(b))
```

```
c = {1:'a', 2:'b', 3:'c', 4:'mohamed'}  
print(type(c))
```

```
d = (1, 2, 3, 'mohamed')  
print(type(d))
```

```
<class 'list'>  
<class 'set'>  
<class 'dict'>  
<class 'tuple'>
```

Si le changement se fait sur la même variable = mutable

Si le changement résulte en une autre variable = immutable

On peut consulter ceci en utilisant la fonction prédéfinie : `id ( )` qui donne un identifiant unique de chaque variable, qui est l'adresse mémoire

**Exemple ici
sur les listes**

a
[1, 2.5, 3, 'mohamed']
id(a)
1896057506112
a[0] = 22
a
[22, 2.5, 3, 'mohamed']
id(a)
1896057506112

Tuples et chaines de caractères = immutable

Peut être changé , mais en créant une variable complètement différentes

```
d  
  
(1, 2, 3, 'mohamed')
```

```
id(d)  
  
1896057201072
```

```
d[0] = 22  
d
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[8], line 1  
----> 1 d[0] = 22  
      2 d
```

TypeError: 'tuple' object does not support item assignment

```
d = list(d)  
d[0] = 22  
d  
  
[22, 2, 3, 'mohamed']
```

```
id(d)  
  
1896078608000
```

```
e = "ab cdef"  
e
```

```
'ab cdef'
```

```
id(e)  
  
2148641436272
```

```
e[0] = 'n'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 e[0] = 'n'
```

TypeError: 'str' object does not support item assignment

```
e = e.replace('c', 'q')  
e
```

```
'ab qdef'
```

```
id(e)  
  
2148642645872
```

Attention à l'effet d'Alias

Lorsque c'est mutable

Aliasing :

```
a = [1, 2, 3, 4]
id(a)
```

1977992713600

```
b = a
id(b)
```

1977992713600

```
b.append(22)
b
```

[1, 2, 3, 4, 22]

```
a    ## j'ai changé dans b,
      ## pourtant a vient d'être changé aussi !!!
      ## c'est l'effet secondaire d'être mutable
```

[1, 2, 3, 4, 22]

même pour les autres données mutables :

```
aa = {1, 2, 3}
id(aa)
```

1977992219328

```
bb = aa
id(bb)
```

1977992219328

```
bb.add(22)
bb
```

{1, 2, 3, 22}

```
aa
{1, 2, 3, 22}
```

solution --> clonning (faire une copie)

```
a = [1, 2, 3]
b = a.copy()
```

```
id(a)
```

1978000095360

```
id(b)
```

1978000086848

```
## Très bien,
## maintenant a et b ne sont pas les mêmes,
## mais ils ont les mêmes valeurs
```


Exemple sur les sets et les dicts

```
b
```

```
{1, 2, 3, 'mohamed'}
```

```
id(b)
```

```
1896057014528
```

```
b[0] = 22
```

```
b
```

TypeError Traceback (most recent call last)

Cell In[13], line 1

```
----> 1 b[0] = 22  
      2 b
```

TypeError: 'set' object does not support item assignment

```
b.add(22)
```

```
b
```

```
{1, 2, 22, 3, 'mohamed'}
```

```
id(b)
```

```
1896057014528
```

```
c
```

```
{1: 'a', 2: 'b', 3: 'c', 4: 'mohamed'}
```

```
id(c)
```

```
1896057518400
```

```
c[1] = 22
```

```
c
```

```
{1: 22, 2: 'b', 3: 'c', 4: 'mohamed'}
```

```
id(c)
```

```
1896057518400
```

Structures de données

lists, set, dict, tuple,

Accès aux valeurs

Accès par indice, par clé, etc.

Liste, tuple, set, et dictionnaire

indexation



```
'''  
lists, set, dictionnaire      = Mutable  
chaines de caractère et tuple = Immutable  
'''
```

```
a = [1, 2.5, 3, 'mohamed']  
print(type(a))
```

```
b = {1, 2, 3, 'mohamed', 2, 3}  
print(type(b))
```

```
c = {1:'a', 2:'b', 3:'c', 4:'mohamed'}  
print(type(c))
```

```
d = (1, 2, 3, 'mohamed')  
print(type(d))
```

```
<class 'list'>  
<class 'set'>  
<class 'dict'>  
<class 'tuple'>
```

```
idx = 0  
a[idx]
```

1

```
b[idx]  #Le set n'a pas d'ordre, donc, ne peut pas être indexé
```

TypeError Traceback (most recent call last)

Cell In[27], line 1

```
----> 1 b[idx]  #le set n'a pas d'ordre, donc, ne peut pas être indexé
```

TypeError: 'set' object is not subscriptable

```
c[idx]
```

KeyError Traceback (most recent call last)

Cell In[29], line 1

```
----> 1 c[idx]
```

KeyError: 0

```
key = 1  
c[key]
```

'a'

```
d[idx]
```

1

Accès aux valeurs

Une seule valeur à la fois (l'indexation)

Strings, listes, tuples (par indice)

```
a = "je suis à la FSSM"
```

```
a[0]    #par indice
```

```
'j'
```

```
a = [3, 15, 10, 1, 2, 4]
```

```
a[2]
```

```
10
```

```
a = (3, 15, 10, 1, 2, 4)
```

```
a[-1]   #-1 veut dire dernier élément
```

```
4
```

Sets (convertir vers liste puis accès par indice)

```
a = {3, 15, 10, 1, 2, 4}
```

```
# Convertir à une liste
```

```
a_list = list(a)
```

```
# maintenant accès par indice
```

```
a_list[1]
```

```
2
```

Les dicts (par clé)

```
a = {1:'a', 2:'b', 3:'c', 4:'mohamed', 'Hello': 10}
```

```
a[1]    #accès par clé
```

```
'a'
```

```
a['Hello']
```

```
10
```

Accès aux valeurs

Plusieurs valeurs à la fois (le slicing)

Strings, listes, tuples (par indice)
a[start : end : step]

```
a = "je suis à la FSSM"
```

```
#extraire 'suis'  
a[3:7:1]
```

```
'suis'
```

```
#extraire 'suis'  
a[3:7] #par défaut, step = 1 s'il n'est pas indiqué
```

```
'suis'
```

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
a[0:9:2] #extraire que les nombre pairs
```

```
[0, 2, 4, 6, 8]
```

Sets (convertir vers liste
puis accès comme les listes)

Les dicts (par clé)
On utilise la fonction prédéfinis : map

```
a = {1:'a', 2:'b', 3:'c', 4:'mohamed', 'Hello': 10}
```

```
keys_to_get = [2, 3, 'Hello']  
values_list = list(map(a.get, keys_to_get))  
print(values_list)
```

```
['b', 'c', 10]
```

Si vous avez une fonction, et que vous voulez l'appliquer sur plusieurs valeurs, c'est ça le rôle de la fonction prédéfinie map

Structures de données

lists, set, dict, tuple,
Méthodes (ou fonctions internes)

Méthodes (ou fonctions internes)

Les strings

Point et tapez sur tabulation pour afficher les différentes méthodes de la structure de données

```
a = "je suis à la FSSM"
```

a.

- capitalize
- casefold
- center
- count
- encode
- endswith
- expandtabs
- find
- format
- format_map

```
a.capitalize()
```

```
'Je suis à la fssm'
```

```
a.replace("M", "m")
```

```
'je suis à la FSSm'
```

```
a.endswith('M')
```

```
True
```

```
a.find('u')
```

```
4
```

```
a.lower()
```

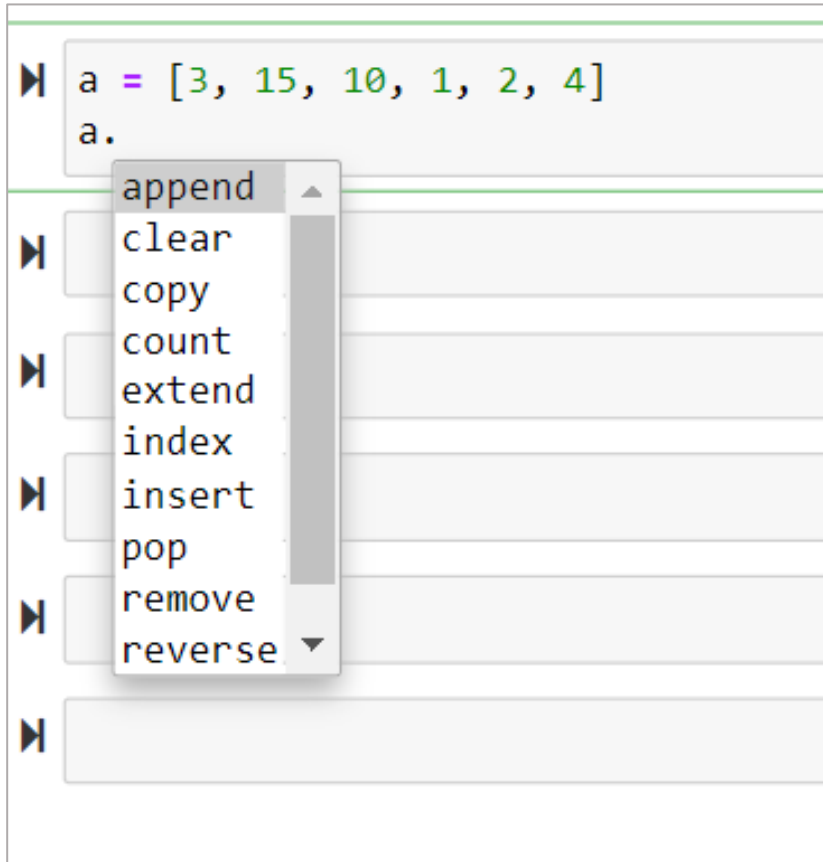
```
'je suis à la fssm'
```

```
a.upper()
```

```
'JE SUIS À LA FSSM'
```

Méthodes (ou fonctions internes)

Les listes



Point et tapez sur tabulation pour afficher les différentes méthodes de la structure de données

```
a = [3, 15, 10, 1, 2, 4]
a.append(18)
```

a

[3, 15, 10, 1, 2, 4, 18]

```
a = [3, 15, 10, 1, 2, 4]
print(a.count(15)) #nombre d'occurence
print(a.count(11))
```

1

0

```
a = [3, 15, 10, 1, 2, 4]
a.sort()
```

a

[1, 2, 3, 4, 10, 15]

```
a = [3, 15, 10, 1, 2, 4]
a.clear()
```

a

[]

```
a = [3, 15, 10, 1, 2, 4]
a.remove(3)
```

a

[15, 10, 1, 2, 4]

Méthodes (ou fonctions internes)

Les tuples (que deux méthodes simples)

```
a = (3, 15, 10, 1, 2, 4)
a.
```

count

index

```
a = (3, 15, 10, 1, 2, 4)
a.index(3)

0

a.index(15)

1

a.index(1)

3
```

```
a = (3, 15, 10, 1, 2, 4)
a.count(15)

1

a.count(200)

0
```

```
a = (3, 15, 10, 1, 2, 4, 1)    ## s'il y a un élément qui se répète
a.index(1)                    ## il retourne l'indice du premier

3
```

Méthodes (ou fonctions internes)

Les sets

```
a = {3, 3, 15, 15, 10, 1, 2, 4}  ## Le set ne peut contenir que des éléments uniques  
a
```

```
{1, 2, 3, 4, 10, 15}
```

```
a = {3, 15, 10, 1, 2, 4}  
a.
```

- add
- clear
- copy
- difference
- difference_update
- discard
- intersection
- intersection_update
- isdisjoint
- issubset

```
a.add(200)
```

```
a
```

```
{1, 2, 3, 4, 10, 15, 200}
```

```
a
```

```
{1, 2, 3, 4, 10, 15}
```

```
b = {4, 10, 20, 30}  
a.intersection(b)
```

```
{4, 10}
```

```
b = {0, 1, 2, 3, 4, 5, 6, 10, 15, 20}  
a.issubset(b)
```

```
True
```

```
b = {0, 2, 3, 4, 5, 6, 10, 15, 20}  
a.issubset(b)
```

```
False
```

```
a
```

```
{1, 2, 3, 4, 10, 15}
```

```
b = {4, 10, 20, 30}  
a.union(b)
```

```
{1, 2, 3, 4, 10, 15, 20, 30}
```

Méthodes (ou fonctions internes)

Les dicts

```
a = {1:'a', 2:'b', 3:'c', 4:'mohamed', 'Hello': 10}
```

▶ a.

- clear
- copy
- fromkeys
- get
- items
- keys
- pop
- popitem
- setdefault
- update

```
a.get(1)
```

```
'a'
```

```
a.keys()
```

```
dict_keys([1, 2, 3, 4, 'Hello'])
```

```
a.values()
```

```
dict_values(['a', 'b', 'c', 'mohamed', 10])
```

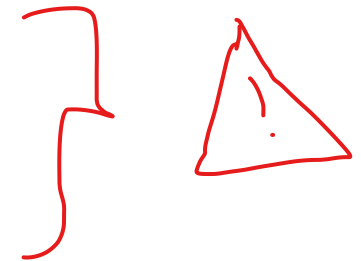
```
a.pop(1)
```

```
a
```

```
{2: 'b', 3: 'c', 4: 'mohamed', 'Hello': 10}
```

```
a.items()
```

```
dict_items([(1, 'a'), (2, 'b'), (3, 'c'), (4, 'mohamed'), ('Hello', 10)])
```



Structure conditionnelle

if , elif, else

Structure conditionnelle

Multi-conditions avec que des 'if'

Condition

```
▶ a = 10
  if a == 10:
      print('Oui')
  else:
      print('Non')
```

Oui

Multi-conditions

```
▶ # if elif else
  a = 7

  if a == 0:
      print("aaa")
  elif a == 1:
      print("bbb")
  elif a == 2:
      print("ccc")
  else:
      print('autre')
```

autre

```
▶ ## pourquoi elif et pas if if if

number = 3

if number < 0:
    print("Le nombre est negative.")
if number == 0:
    print("Le nombre est zero.")
if 0 < number <= 100:
    print("Le nombre est entre 1 et 100.")
if 100 < number <= 1000:
    print("Le nombre est entre 101 et 1000.")
else:
    print("Le nombre est supérieur à 1000.")
```

C'est quoi Le problème ici !!!

Le nombre est entre 1 et 100.

Le nombre est supérieur à 1000.

Structure conditionnelle

Multi-conditions avec que des 'if'

Problème

► *## pourquoi elif et pas if if if*

```
number = 3

if number < 0:
    print("Le nombre est negative.")
if number == 0:
    print("Le nombre est zero.")
if 0 < number <= 100:
    print("Le nombre est entre 1 et 100.")
if 100 < number <= 1000:
    print("Le nombre est entre 101 et 1000.")
else:
    print("Le nombre est supérieur à 1000.")
```

C'est quoi Le problème ici !!!

Le nombre est entre 1 et 100.
Le nombre est supérieur à 1000.

Multi-conditions avec des 'if' et 'elif'

La version correcte

►

*## Dans l'exemple précédent, La dernière else ne dépend que de if précédente
La version correcte doit être comme suit :*

```
number = 3

if number < 0:
    print("Le nombre est negative.")
elif number == 0:
    print("Le nombre est zero.")
elif 0 < number <= 100:
    print("Le nombre est entre 1 et 100.")
elif 100 < number <= 1000:
    print("Le nombre est entre 101 et 1000.")
else:
    print("Le nombre est supérieur à 1000.")
```

Le nombre est entre 1 et 100.

Opérateurs logiques

Souvent utilisés avec les structures conditionnelles

Opérateurs logiques

1. `and` : Renvoie True si les deux opérandes sont True.

```
In [24]: ► True and True # True
```

```
Out[24]: True
```

```
In [25]: ► True and False # False
```

```
Out[25]: False
```

2. `or` : Renvoie True si au moins l'une des opérandes est True.

```
In [27]: ► True or False # True
```

```
Out[27]: True
```

```
In [28]: ► False or False # False
```

```
Out[28]: False
```

3. `==` : Opérateur d'égalité, renvoie True si les opérandes sont égales.

```
In [31]: ► 5 == 5 # True
```

```
Out[31]: True
```

```
In [32]: ► 5 == 10 # False
```

```
Out[32]: False
```

4. `!=` : Opérateur de non-égalité, renvoie True si les opérandes ne sont pas égales.

```
In [33]: ► 5 != 10 # True
```

```
Out[33]: True
```

```
In [34]: ► 5 != 5 # False
```

```
Out[34]: False
```


Opérateurs logiques

5. `<` : Opérateur de moins que, renvoie True si l'opérande de gauche est inférieur à l'opérande de droite.

```
In [35]: 5 < 10 # True
```

```
Out[35]: True
```

```
In [36]: 10 < 5 # False
```

```
Out[36]: False
```

6. `<=` : Opérateur de moins ou égal à, renvoie True si l'opérande de gauche est inférieur ou égal à l'opérande de droite.

```
In [38]: 5 <= 5 # True
```

```
Out[38]: True
```

```
In [37]: 10 <= 5 # False
```

```
Out[37]: False
```

Opérateurs logiques

7. `>` : Opérateur de plus que, renvoie True si l'opérande de gauche est supérieur à l'opérande de droite.

```
In [39]: 10 > 5 # True
```

```
Out[39]: True
```

```
In [40]: 5 > 10 # False
```

```
Out[40]: False
```

8. `>=` : Opérateur de plus ou égal à, renvoie True si l'opérande de gauche est supérieur ou égal à l'opérande de droite.

```
In [41]: 5 >= 5 # True
```

```
Out[41]: True
```

```
In [42]: 5 >= 10 # False
```

```
Out[42]: False
```

Opérateurs logiques

9. `in` : Opérateur d'appartenance, renvoie `True` si une valeur est trouvée dans la séquence.

```
In [43]: 5 in [1, 2, 3, 4, 5] # True
```

```
Out[43]: True
```

```
In [44]: 10 in [1, 2, 3, 4, 5] # False
```

```
Out[44]: False
```

10. `not` : Renvoie `True` si l'opérande est `False`, et vice versa.

```
In [29]: not True # False
```

```
Out[29]: False
```

```
In [30]: not False # True
```

```
Out[30]: True
```

11. `not in` : Opérateur de négation d'appartenance, renvoie `True` si une valeur n'est pas trouvée dans la séquence.

```
In [45]: 10 not in [1, 2, 3, 4, 5] # True
```

```
Out[45]: True
```

```
In [46]: 5 not in [1, 2, 3, 4, 5] # False
```

```
Out[46]: False
```

Structure répétitive

La boucle while

+

4 types de boucle for

while VS for

Boucle while

- Utilisée lorsque vous ne savez pas combien d'itérations seront nécessaires à l'avance et que vous souhaitez répéter le bloc de code tant qu'une condition est vraie.
- La syntaxe générale est :
 while condition:
 ## block de code à répéter
- La boucle while continue tant que la condition spécifiée est vraie. Si la condition devient fausse, la boucle s'arrête.

Boucle for :

- Utilisée lorsque vous avez une séquence d'éléments sur laquelle itérer, telle qu'une liste, un tuple, une chaîne de caractères, etc.
- Pour la syntaxe, il existe 4 méthodes de faire une boucle for :
 nous allons les voir dans les slides suivants
- La boucle for itère sur chaque élément de la séquence jusqu'à ce qu'il n'y ait plus d'éléments.

La boucle while

► *## boucle infinie: Attention !!*

```
# x = 1
# while x==1:
#     print("aaaaa")
```

Lors de l'utilisation de la boucle while, toujours s'assurer bien que la condition deviendra « True » dans un moment spécifique, sinon vous aurez une boucle infinie

```
## Exemple: programme qui calcule la somme jusqu'à atteindre une limite

limit = 100
current_sum = 0
number = 1

while (current_sum + number) <= limit:    #ne pas faire ==
    print(number, current_sum)
    current_sum += number
    number += 1

print("\nLa somme jusqu'à la limite", limit, "est:", current_sum)
```

```
1 0
2 1
3 3
4 6
5 10
6 15
7 21
8 28
9 36
10 45
11 55
12 66
13 78
```

La somme jusqu'à la limite 100 est: 91

Boucle for, méthode 1

```
for i in range(a, b):  
    #code à répéter
```

```
▶ for i in range(3, 12):  
    print(i)
```

```
3  
4  
5  
6  
7  
8  
9  
10  
11
```

```
▶ n = 10  
for i in range(n):    #si a n'est pas spécifiée, par défaut c'est zéro  
    print(i)
```

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

Note:

a inclut, mais b n'est pas inclut

Boucle for, méthode 2

for élément in itérable:
#code à répéter



```
► mylist = [1, 3, 6, 8, 10]
  for element in mylist:
    res = element + 8
    print(res)
```

9
11
14
16
18

```
► s = "je suis à la FSSM"
  for lettre in s:
    print(lettre)
```

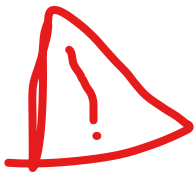
j
e

s
u
i
s

à

l
a

F
S
S
M



Boucle for, méthode 3

for i, élément in enumerate(itérable):
#code à répéter

```
► mylist = [22, 3, 6, 8, 10]
for i, element in enumerate(mylist):
    print(i, element)
```

```
0 22
1 3
2 6
3 8
4 10
```

L'indice et l'élément

```
► s = "je suis à la FSSM"
for i, lettre in enumerate(s):
    print(i, lettre)
```

```
0 j
1 e
2
3 s
4 u
5 i
6 s
7
8 à
9
10 l
11 a
12
13 F
14 S
15 S
16 M
```

Boucle for, méthode 4

`for` élément_1, élément_2 `in` zip(itérable_1, itérable_2):
#code à répéter

```
letters = ['a', 'b', 'c', 'd', 'e']  
numbers = [1, 2, 3, 4, 5]
```

```
for L, N in zip(letters, numbers):  
    print(L, N)
```

```
a 1  
b 2  
c 3  
d 4  
e 5
```

► *## si une des deux listes est petite que l'autre,
la boucle s'arrêtera lorsqu'on atteind
la taille de la petite liste*

```
letters = ['a', 'b', 'c', 'd', 'e']  
numbers = [0, 1, 2]
```

```
for L, N in zip(letters, numbers):  
    print(L, N)
```

```
a 0  
b 1  
c 2
```

Input de l'utilisateur

La fonction input()

Interaction avec l'utilisateur via : `input('message')`

```
In [*]: ▶ a = input("entrer la valeur de a:")
        entrer la valeur de a: 33

In [5]: ▶ a
Out[5]: '33'
```

```
type(a)
str

a = int(a)
type(a)
int
```

Note:

- lorsque j'affiche a, il l'affiche comme chaîne de caractère
- Donc, si vous voulez faire des opérations numériques sur a, vous devez d'abord la convertir en type numérique (par exemple 'int')

La fonction print

Affichage plus personnalisé

Différentes méthodes pour faire l'affichage print()

1. **Utilisation de la fonction `print`** : La manière la plus basique d'afficher du texte en Python est d'utiliser la fonction `print`.

```
▶ print("Bonjour")
```

Bonjour

2. **Concaténation de chaînes de caractères** : Vous pouvez concaténer des chaînes de caractères à l'aide de l'opérateur `+` et ensuite afficher le résultat.

```
▶ salutation = "Bonjour"  
nom = "le monde"  
print(salutation + ", " + nom + " !")
```

Bonjour, le monde !

Différentes méthodes pour faire l'affichage print()

3. **F-strings (chaînes de caractères formatées)** : Introduites dans Python 3.6, les f-strings offrent une manière concise d'insérer des expressions à l'intérieur des chaînes de caractères.

►

```
nom = "Ahmed"
print(f"Bonjour, {nom} !")
```

Bonjour, Ahmed !

►

```
x = 5
print(f"Le carré de {x} est {x**2}")
```

Le carré de 5 est 25

4. **Multiples arguments avec la fonction print** : La fonction `print` peut prendre plusieurs arguments, séparés par des virgules.

►

```
salutation = "Bonjour"
nom = "Ahmed"
print(salutation, ",", nom, "!")
```

Bonjour , Ahmed !

Différentes méthodes pour faire l'affichage print()

5. **Affichage de variables avec différents types de données** : Vous pouvez afficher des variables de différents types en les séparant par des virgules.

```
► age = 25
  taille = 1.75
  print("Âge :", age, "ans, Taille :", taille, "mètres")
```

Âge : 25 ans, Taille : 1.75 mètres

6. **Spécifier nbr de chiffres après la virgule:**

```
► nbr = 3.14159
  format_nbr = "{:.2f}".format(number)
  print(format_nbr)
```

3.14

```
► ## ou
  nbr = 3.14159
  format_nbr = f"{number:.2f}"
  print(format_nbr)
```

3.14



Différentes méthodes pour faire l'affichage print()

7. **Affichage sur la même ligne avec le paramètre `end`** : Par défaut, la fonction `print` ajoute un caractère de saut de ligne à la fin. Vous pouvez changer ce comportement en utilisant le paramètre `end`.

```
➤ print("Ceci est sur la même ligne", end=" ")  
print("que la ligne précédente.")
```

Ceci est sur la même ligne que la ligne précédente.

```
➤ print("Ceci n'est sur la même ligne") #par défaut end= '\n'  
print("que cette ligne.")
```

Ceci n'est sur la même ligne
que cette ligne.